

Plan Composition

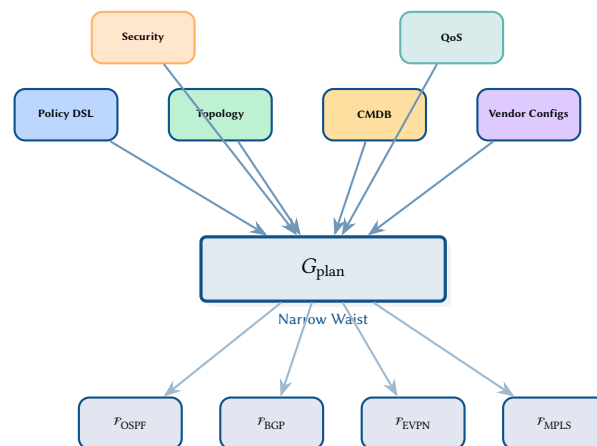
From Diverse Intent Sources to a Unified Plan Topology

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. The plan topology G_{plan} , defined in ANK-TR-2026-03 and catalogued in ANK-TR-2026-04, serves as a vendor-neutral intermediate representation for all network protocol state. This report addresses the *composition problem*: how is a plan topology constructed from the diverse sources of intent that network engineers actually work with? We present a **narrow-waist architecture** in which many input modalities—high-level policy DSLs, topology fragments, source-of-truth databases, vendor configuration extraction, security segmentation policies, and QoS specifications—are compiled and merged into the single canonical graph G_{plan} . We define a **fragment algebra** for composing partial plan graphs with well-defined union, overlay, and override operators, and establish conditions under which fragment composition is associative and commutative. A **policy compilation** framework maps high-level intent statements to the flat attribute schema of the plan topology. The composition pipeline—parse, normalise, merge, allocate (Thunks), default, and validate—is specified formally and illustrated with practical operator workflows anchoring GitOps CI/CD pipelines to NSoT inventory realities. The paper provides end-to-end worked examples covering greenfield data-centre fabric design, brownfield migration from existing vendor configurations, and multi-domain network. The narrow-waist principle ensures that the downstream multi-layer model and network config compiler remain decoupled from the diversity of upstream intent sources, enabling independent evolution of both input tooling and implementation rendering.

Contents

1	Introduction	3
1.1	The Composition Problem	3
1.2	The Narrow-Waist Principle	3
1.3	Contributions	4
1.4	Document Roadmap	4
2	The Narrow-Waist Architecture	4
2.1	Architectural Layers	4
2.2	Formal Definition of Composition	5
2.3	Composition Invariants	5
2.4	The Hourglass Analogy	6
2.5	Relationship to Companion Reports	7
3	Input Modalities	7
3.1	Topology Fragments	7
3.2	Design Templates	7
3.3	High-Level Policy Intent	9
3.4	Source-of-Truth Databases	9
3.5	Vendor Configuration Extraction	9
3.6	Security Segmentation Policy	10
3.7	QoS Specifications	11
3.8	Compliance and Regulatory Constraints	12
3.9	Modality Summary	12
4	The Composition Pipeline	12
4.1	Architectural Framing: The Narrow Waist	13
4.2	Mathematical Foundations	13
4.3	Stage 1: Parse	13
4.4	Stage 2: Normalise	13
4.5	Stage 3: Merge	14
4.6	Stage 4: Allocate	14
4.7	Stage 5: Default	16
4.8	Stage 6: Validate	16
4.9	Case Study: Tracing Intent through the Pipeline	17
4.10	End-to-End Attribute Lifecycle	17
4.11	Pipeline Properties	17
5	Fragment Algebra	18
5.1	Plan Fragments as Partial Graphs	18
5.2	The Union Operator ($\cup$)	18
5.3	The Overlay Operator ($\triangleright$)	19
5.4	The Override Operator ($\gg$)	19
5.5	Composition Order and the Fragment DAG	20
5.6	Algebraic Laws	20
5.7	Categorical Properties of Composition	20
5.8	Join-Semilattice Structure	21
6	Policy Compilation	21
6.1	Design Rules: Primitives and Assertions	21
6.2	Policy Language Concepts	22
6.3	Match Predicates	22
6.4	Attribute Generators	22
6.5	Policy Evaluation Order	23
6.6	Policy Composition Patterns	23
6.7	Firewall and ACL Policy Compilation	25
6.8	Route Policy Compilation	26
6.9	Micro-Segmentation and SGT Policies	27
6.10	Compilation Complexity	27
7	Source-of-Truth Integration	28
7.1	Schema Mapping	28
7.2	Incremental Synchronisation	28

7.3	Drift Detection	28
8	Operator Workflows and Tractability	29
8.1	The Role of the DSL	29
8.2	NSoT and DSL Integration Flow.	30
9	Worked Examples	30
9.1	Example 1: Greenfield Data Centre Fabric.	30
9.2	Example 2: Brownfield Migration.	31
9.3	Example 3: Multi-Domain Composition.	33
9.4	Example 4: Day 2 Operations - Security Remediation.	34
9.5	Example 5: Hardware Refresh / Vendor Swap	35
10	Validation and Verification	35
10.1	Composition-Specific Validation Rules.	35
10.2	Validation Report Structure	36
10.3	Provenance Trace Visualisation	36
10.4	Iterative Composition	37
11	Implementation Considerations	37
11.1	Architecture Overview	37
11.2	Performance Considerations.	38
11.3	Extensibility.	38
11.4	Configuration Management	38
12	Critical Assessment: Risks and Challenges	39
12.1	The Complexity of Fragment Algebra	39
12.2	The “Everything is an Attribute” Fallacy	39
12.3	Conflict Resolution and Priority Creep	39
12.4	The Verification Gap	39
12.5	Provenance and Metadata Overhead	40
13	Related Work	40
13.1	Intent-Based Networking and Configuration Synthesis	40
13.2	Topological Specification and Structural Intent	40
13.3	Network Verification vs. Constructive Composition	40
13.4	Standardized Data Models (IETF and OpenConfig)	41
13.5	Category Theory and Graph Transformation	41
14	Conclusion	42
14.1	Summary	42
14.2	The Narrow Waist in Practice.	42
14.3	Relationship to the Report Series	42
14.4	Future Work	42
	Revision History	44
CMDB	Configuration Management Database	9
SoT	Source of Truth	9
DSL	Domain-Specific Language	29
NTE	Network Topology Engine	37

Contents

1 Introduction

1.1 The Composition Problem

The companion reports ANK-TR-2026-03 and ANK-TR-2026-04 define the plan topology G_{plan} as a vendor-neutral, flat-attribute graph that captures the complete intended state of a network. The functorial overlay mappings $\mathcal{F}$ consume this graph to produce protocol-specific views: OSPF adjacencies, BGP sessions, EVPN fabrics, and more.

But where does the plan come from?

The What and the Why: Bridging Intent to Mathematics. While the formal plan topology represents a pristine mathematical end-state (The What), operational network engineering is inherently messy and distributed (The Why). Engineers do not author property hypergraphs directly. Instead, they interact with the network through diverse, often conflicting abstractions: high-level policies, IPAM databases, spreadsheet allocations, and legacy vendor templates.

In this report, we frame the composition problem through the lens of **Category Theory**. We define a **Category of Plan Graphs (Plan)**, where objects are plan topologies and morphisms represent structural refinements. The composition pipeline is then a series of **functorial transformations** that preserve structural invariants while merging disparate intent fragments. This rigorous framing allows us to move beyond "scripts that generate config" to mathematical transformations that guarantee correctness-by-construction.

Fundamentally, this composition pipeline represents an evolution of the network engineer's role—transitioning them from "artisanal CLI typists" who memorize vendor syntax, to "architects of mathematical blueprints" who manipulate abstract intent structures.

In practice, no single input format captures the full breadth of network intent. Operators express their requirements through a patchwork of artefacts:

- High-level policy statements: "all spine routers shall run iBGP with route-reflector clients on every leaf."
- Topology descriptions: "a 3-stage Clos fabric with 4 spines and 16 leaves."
- Source-of-truth databases: IP address management, device inventories, VLAN registries.
- Existing vendor configurations: the running state of a brownfield network that must be captured and normalised.
- Security segmentation policies: micro-segmentation rules, firewall zone definitions, SGT assignment matrices.
- QoS specifications: traffic class definitions, bandwidth guarantees, latency SLA targets.
- Compliance constraints: regulatory requirements mandating encryption, access control, or audit logging on specific links.

Each of these artefacts is expressed in a different language, at a different level of abstraction, and by a different stakeholder. The **composition problem** is: how do we compile, merge, and validate these heterogeneous inputs into a single, consistent G_{plan} ?

1.2 The Narrow-Waist Principle

Our answer is a **narrow-waist architecture** (fig. 1), directly inspired by the Internet protocol hourglass [1]. Just as IP provides a single, universal datagram service between diverse link technologies below and diverse applications above, G_{plan} provides a single, universal representation of network intent between diverse input modalities and the downstream multi-layer model and configuration compiler.

The narrow waist has three critical properties:

Design Rule 1: Input Plurality

Any number of input modalities may contribute to a plan topology. The plan topology schema imposes no constraints on *how* attributes are produced, only on *what* attributes must satisfy after composition.

Design Rule 2: Output Decoupling

Downstream consumers (functors, renderers, validators) depend only on the plan topology schema, never on the input modalities. Adding a new input source requires no changes to any functor.

Design Rule 3: Composability

The plan topology can be assembled incrementally from independent fragments. Each fragment contributes a subset of nodes, endpoints, edges, or attributes; the composition pipeline merges them into a single consistent graph.

Insight:
The plan topology is the what; the functors are the how. This report addresses the where from.

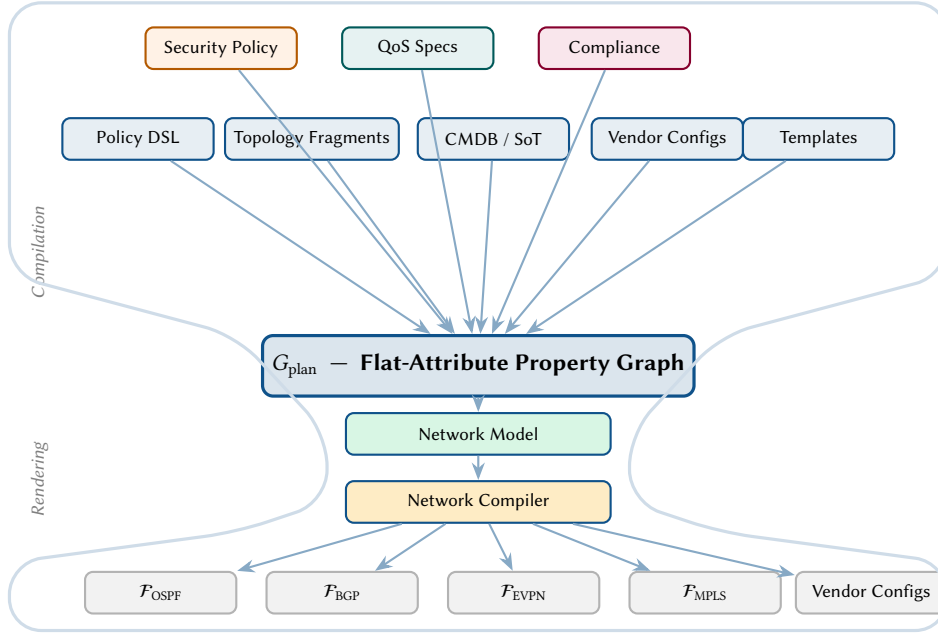


Figure 1. The narrow-waist architecture. Many diverse input modalities (top) are compiled into a single canonical plan topology (centre), which is then projected into a multi-layer network model and compiled into platform-specific configuration state (bottom). The plan topology serves as the *narrow waist*, analogous to IP in the Internet protocol hourglass.

1.3 Contributions

This report makes the following contributions:

1. A **narrow-waist architecture** for network intent composition, with G_{plan} as the canonical intermediate representation (section 2).
2. A taxonomy of **input modalities** that feed the plan topology, with concrete syntax examples for each (section 3).
3. A formal **composition pipeline** with six stages—parse, normalise, merge, allocate, default, validate—that transforms heterogeneous inputs into a valid plan topology (section 4).
4. A **fragment algebra** defining union, overlay, and override operators on partial plan graphs, with algebraic properties (section 5).
5. A **policy compilation** framework that maps high-level intent to flat plan attributes (section 6).
6. A framework for **operator workflows and tractability**, bridging GitOps intent with NSoT reality (section 8).
7. Three **end-to-end worked examples**: greenfield DC fabric, brownfield migration, and multi-domain composition (section 9).

1.4 Document Roadmap

Section 2 formalises the narrow-waist architecture and its invariants. Section 3 catalogues the input modalities with syntax examples. Section 4 specifies the composition pipeline. Section 5 defines the fragment algebra. Section 6 addresses policy compilation. Section 7 covers source-of-truth integration. Section 8 defines practical operator workflows. Section 9 presents worked examples. Section 10 describes plan validation. Section 11 discusses implementation. Section 13 surveys related work, and section 14 concludes.

2 The Narrow-Waist Architecture

This section formalises the narrow-waist principle and establishes the invariants that any composition pipeline must preserve.

2.1 Architectural Layers

The narrow-waist architecture comprises three layers (fig. 2):

1. **Intent Layer** (above the waist): the diverse input modalities through which operators express network requirements. Each modality has its own syntax, abstraction level, and domain model.
2. **Plan Layer** (the waist): the single canonical representation G_{plan} , a flat-attribute labelled property hypergraph as defined in ANK-TR-2026-03.
3. **Realisation Layer** (below the waist): the functorial overlay mappings $\mathcal{F}_t$ and vendor-specific renderers that consume G_{plan} to produce protocol views and device configurations.

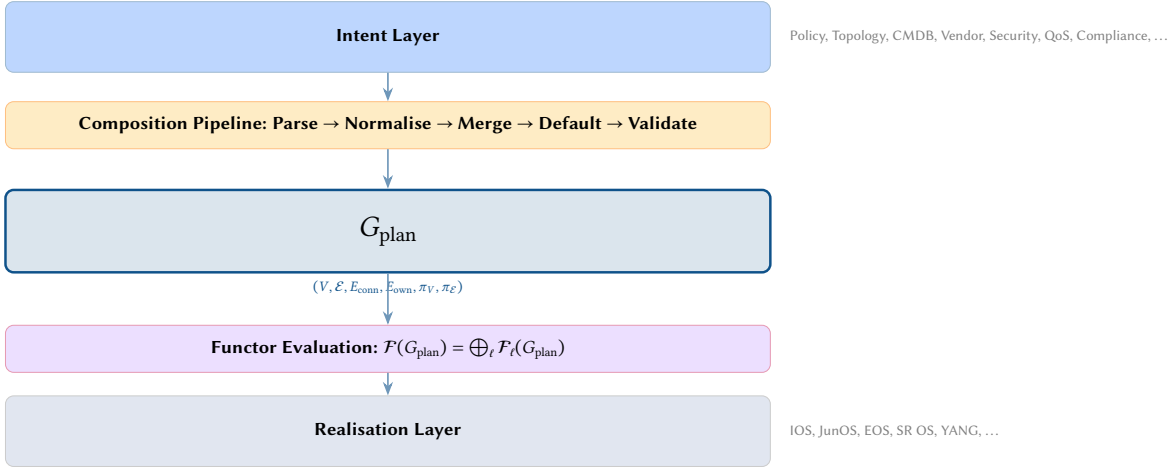


Figure 2. The three-layer narrow-waist architecture. The composition pipeline compiles intent-layer artefacts into G_{plan} ; functorial mappings then realise protocol-specific views and vendor configurations.

2.2 Formal Definition of Composition

Let $S = \{S_1, S_2, \dots, S_k\}$ be a set of *intent sources*, where each S_i produces a *plan fragment* F_i . A plan fragment is a partial plan topology:

$$F_i = (V_i, \mathcal{E}_i, E_{\text{conn}}^i, E_{\text{own}}^i, \pi_V^i, \pi_E^i) \quad (1)$$

where each component is a subset of (or partial function over) the corresponding component in a complete plan topology. Critically, the attribute maps π_V^i and π_E^i may be *partial*: a fragment need not assign every attribute to every node or endpoint.

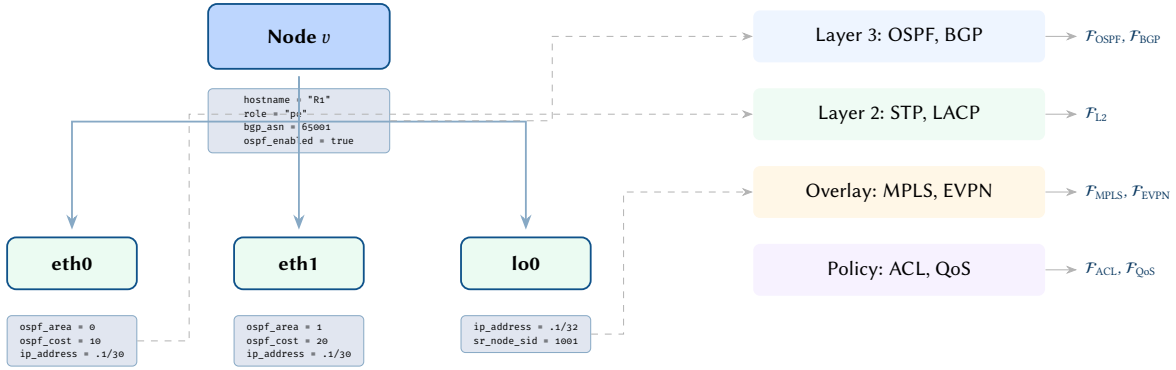


Figure 3. Attribute scope and layer mapping. Node-level attributes (top left) are stored in π_V ; endpoint-level attributes (bottom left) in π_E . Each protocol layer (right) reads from both scopes via its corresponding functor.

Definition 1 (Plan Composition). A plan composition operator $\oplus$ takes a set of fragments and produces a complete plan topology:

$$G_{\text{plan}} = \bigoplus_{i=1}^k F_i = F_1 \oplus F_2 \oplus \dots \oplus F_k \quad (2)$$

subject to the constraint that the result satisfies all structural invariants of G_{plan} (unique ownership, bidirectional connectivity, no orphan nodes) and all protocol consistency rules defined in ANK-TR-2026-04.

2.3 Composition Invariants

The composition operator must preserve the following invariants:

Design Rule 4: Structural Preservation

If each fragment F_i satisfies the structural invariants of G_{plan} on its own subgraph, then $\bigoplus_i F_i$ must also satisfy them on the complete graph. Formally: $\forall i. \text{valid}_{\text{struct}}(F_i) \implies \text{valid}_{\text{struct}}(\bigoplus_i F_i)$.



Figure 4. Structural analogy between the Internet protocol hourglass (left) and the plan composition hourglass (right). The plan hourglass specifically captures the flow from G_{plan} through the multi-layer network model into the network compiler, before reaching vendor-specific outputs.

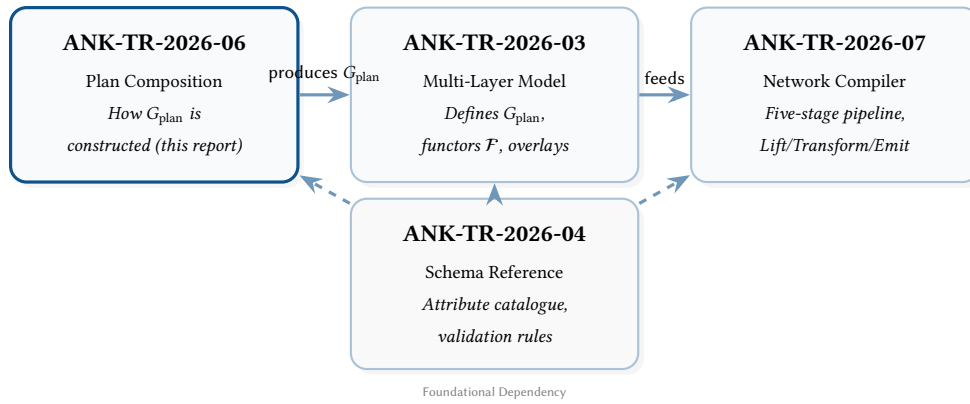


Figure 5. Relationship between the companion reports. The plan is composed (TR-06), modeled as a multi-layer graph (TR-03), and finally compiled into configurations (TR-07). All reports depend on the canonical Schema Reference (TR-04).

Design Rule 5: Attribute Determinism

For any node v or endpoint e in the composed plan, every attribute key maps to exactly one value. If two fragments assign different values to the same key on the same entity, the composition pipeline must either resolve the conflict deterministically or reject the composition with an error.

Design Rule 6: Provenance Traceability

Every attribute value in the composed G_{plan} must be traceable to the intent source that produced it. This enables auditing: for any attribute, an operator can determine which policy, template, or source-of-truth record generated it.

2.4 The Hourglass Analogy

The analogy to the Internet hourglass (fig. 4) is deliberate and instructive:

- **Above the waist:** applications (HTTP, SMTP, DNS) map to intent sources (policies, topologies, inventories). Both are diverse, domain-specific, and evolve independently.
- **At the waist:** IP maps to G_{plan} . Both are deliberately simple, stable, and universal. IP carries datagrams; G_{plan} carries flat key-value attributes on a labelled property graph.
- **Below the waist:** link technologies (Ethernet, Wi-Fi, LTE) map to the realization stages: the **Multi-Layer Network Model** (TR-03) and the **Network Config Compiler** (TR-07). These stages decouple the universal plan from the diverse vendor formats (IOS, JunOS, EOS).

The key architectural benefit is **independent evolution**: new input modalities can be added without modifying any functor, and new vendor renderers can be added without modifying any input parser. The plan topology schema is the contract that decouples the two sides.

Insight:
The narrow waist succeeds precisely because it is boring: a simple, stable, well-defined interface that everyone can agree on. The expressivity lives at the edges, and the complexity is managed by the compiler pipeline below the waist.

2.5 Relationship to Companion Reports

Figure 5 shows how this report relates to the companion documents:

- **ANK-TR-2026-06** (this report) specifies how the plan is composed from diverse intent sources.
- **ANK-TR-2026-03** defines the mathematical structure of the composed plan and specifies how functors consume it.
- **ANK-TR-2026-07** (Network Config Compiler) implements the final stage of the pipeline, translating the model views into vendor-specific configuration state.
- **ANK-TR-2026-04** (Schema Reference) provides the complete attribute catalogue and validation rules used by all other stages.

Together, the reports form a complete end-to-end lifecycle: *compose* (TR-06) → *model* (TR-03) → *compile* (TR-07), all anchored by the canonical *schema* (TR-04).

3 Input Modalities

This section catalogues the input modalities that can contribute to a plan topology. For each modality we describe its purpose, typical syntax, and the plan attributes it produces. Figure 6 provides an overview.

3.1 Topology Fragments

Topology fragments describe the physical and logical structure of the network: devices, interfaces, and their interconnections. They are the most fundamental input modality, as they establish the node and endpoint sets (V and $\mathcal{E}$) that all other modalities annotate.

Topology Fragment — YAML Syntax

```
# Spine-leaf fabric: 2 spines, 4 leaves
nodes:
  - id: spine-01
    role: spine
    platform: arista_7050x
  - id: spine-02
    role: spine
    platform: arista_7050x
  - id: leaf-01
    role: leaf
    platform: arista_7020r
  - id: leaf-02
    role: leaf
    platform: arista_7020r
links:
  - src: spine-01:Ethernet1
    dst: leaf-01:Ethernet49
  - src: spine-01:Ethernet2
    dst: leaf-02:Ethernet49
  - src: spine-02:Ethernet1
    dst: leaf-01:Ethernet50
  - src: spine-02:Ethernet2
    dst: leaf-02:Ethernet50
```

This fragment produces:

- Four nodes in V with `hostname` and `role` attributes.
- Eight endpoints in $\mathcal{E}$ (one per interface per side of each link).
- Four connectivity edges in E_{conn} .
- Eight ownership edges in E_{own} (each device owns its interfaces).

Topology as Foundation

Topology fragments are typically processed first in the composition pipeline because they establish the graph structure that all subsequent fragments annotate. A policy fragment that references `spine-01` requires that node to already exist in the graph.

3.2 Design Templates

Design templates are parameterised topology generators that produce complete plan fragments for common architectures.

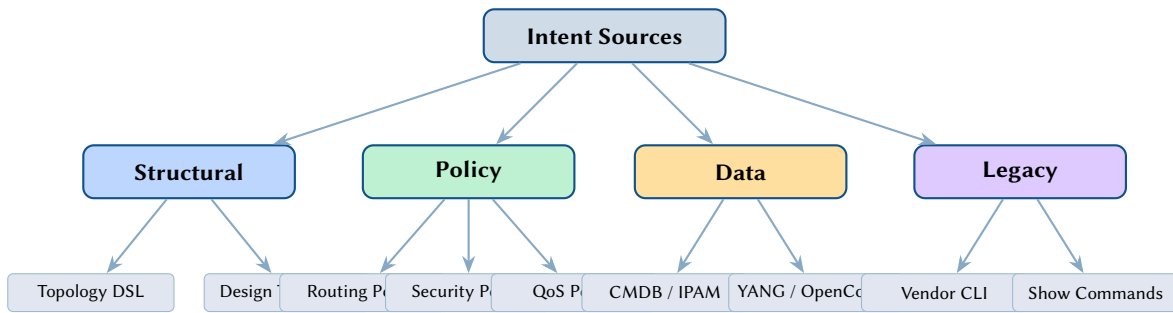


Figure 6. Taxonomy of intent sources grouped into four categories: structural (topology and templates), policy (routing, security, QoS), data (inventories and data models), and legacy (existing configurations and operational state).

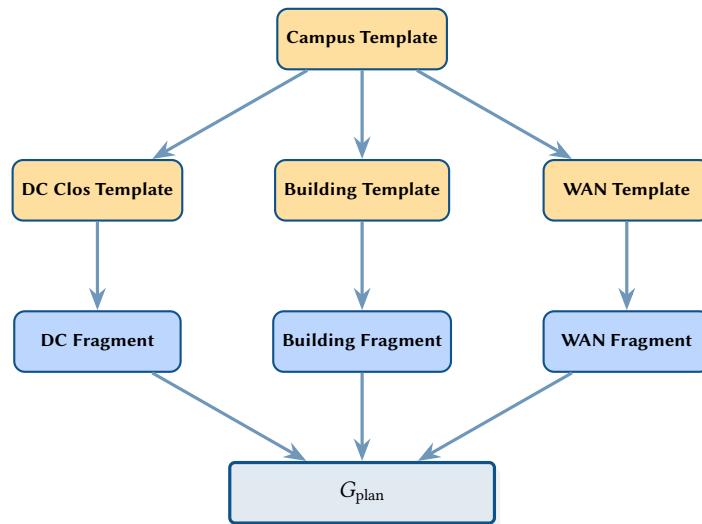


Figure 7. Hierarchical template expansion. A campus template instantiates domain-specific sub-templates, each producing a plan fragment. Fragments are merged into the final plan topology.

Design Template – Clos Fabric Generator

```

template: clos_fabric
parameters:
  spine_count: 4
  leaf_count: 16
  spine_platform: arista_7050x
  leaf_platform: arista_7020r
  spine_asn: 65000
  leaf_asn_base: 65001      # auto-incremented per leaf
  underlay: ospf
  overlay: evpn_vxlan
  loopback_prefix: 10.255.0.0/24
  p2p_prefix: 10.0.0.0/16
  
```

A template expands into a complete plan fragment containing nodes, endpoints, connectivity, and protocol attributes. Templates are composable: a campus template might instantiate a Clos fabric template for the data centre, a collapsed-core template for each building, and a WAN template connecting them.

Operational Power and Trade-offs

The Power of the Option: Templates encapsulate complex, recurring architectural patterns into a single reusable block. They allow an organization to standardize its "gold-standard" architectural specifications.

- **Pros:** Immense scaling power and consistency. Operators can instantiate a massive 5-stage Clos fabric with a handful of parameters, ensuring strict adherence to organizational Design Rules.
- **Cons:** Loss of edge-case flexibility. Templates force uniformity; accommodating unique, bespoke requirements ("snowflake" designs) often requires breaking the template or layering complex override policies on top.

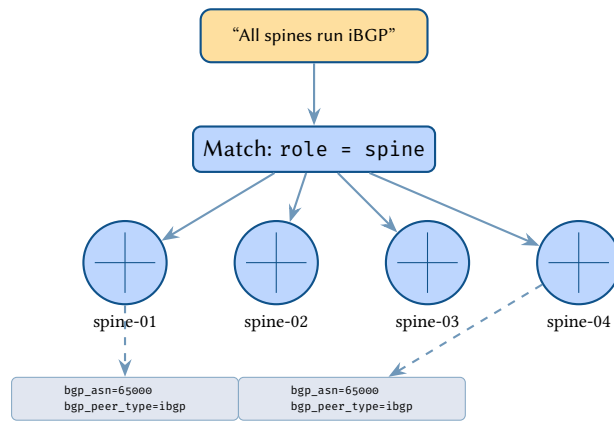


Figure 8. Policy compilation: a high-level intent statement is decomposed into a match predicate and a set of attribute assignments, which are applied to all matching nodes.

3.3 High-Level Policy Intent

Policy intent statements express *what* the network should do without specifying *how* at the device level. They operate at a higher abstraction than direct attribute assignment.

Policy Intent — Routing

```

policies:
- name: spine-ibgp
  description: "All spines form iBGP full mesh"
  match:
    role: spine
  set:
    bgp_asn: 65000
    bgp_peer_type: ibgp
    bgp_peer_group: SPINE_MESH
    bgp_next_hop_self: true
- name: leaf-ebgp-to-spine
  description: "Each leaf peers eBGP with every spine"
  match:
    role: leaf
  set:
    bgp_asn: "auto:65001+index"
    bgp_peer_type: ebgp
    bgp_peer_group: SPINE_UPLINK
  
```

Policy intent is the most expressive input modality because it allows operators to reason about *groups* of devices rather than individual nodes. The policy compiler (section 6) evaluates match predicates against the current graph state to determine which nodes and endpoints receive which attributes.

3.4 Source-of-Truth Databases

Network source-of-truth (Source of Truth (SoT)) systems—such as NetBox, Nautobot, or custom Configuration Management Database (CMDB) platforms—store authoritative records of device inventories, IP address allocations, VLAN registries, and circuit information. These systems are typically the canonical source for:

- **Device inventory:** hostnames, roles, platforms, serial numbers, rack locations.
- **IP address management (IPAM):** prefix allocations, interface IP assignments, VRF membership.
- **VLAN registries:** VLAN IDs, names, and site assignments.
- **Circuit records:** WAN circuit IDs, provider information, bandwidth.
- **Tenancy and tagging:** organisational ownership and classification metadata.

The integration is performed by a *SoT adapter* (fig. 9) that maps the source-of-truth data model to plan topology attributes. Section 7 provides a detailed treatment of schema mapping and incremental synchronisation.

3.5 Vendor Configuration Extraction

In brownfield environments, the existing network already has a rich configuration that must be captured and normalised. Vendor configuration extraction parses device configurations (IOS, JunOS, EOS, SR OS) and produces plan fragments containing the attributes that each configuration implies.

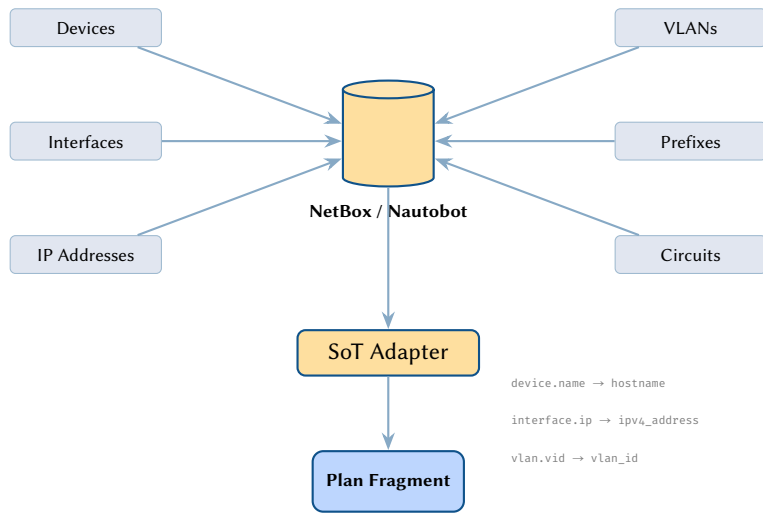


Figure 9. Source-of-truth integration via a schema adapter. The adapter maps SoT data models (devices, interfaces, IPs, VLANs) to plan topology attributes, producing a plan fragment.

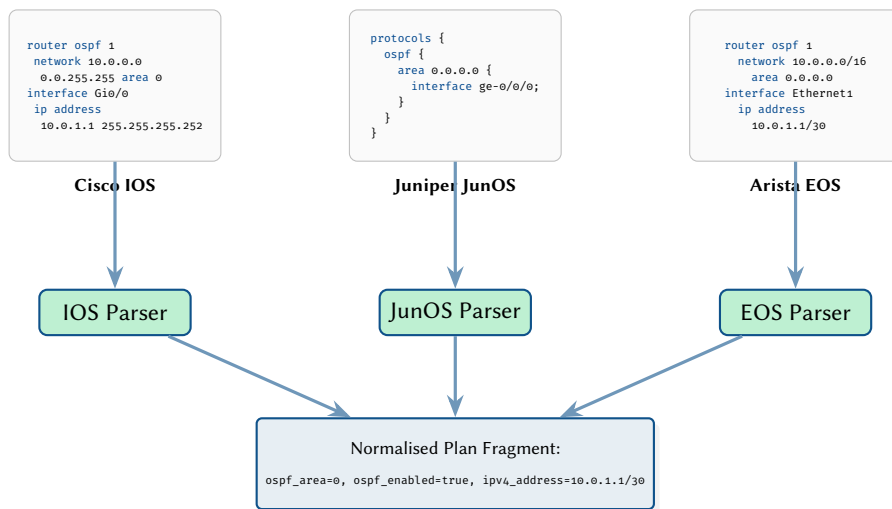


Figure 10. Vendor configuration extraction. Platform-specific parsers normalise diverse CLI syntaxes into a common set of plan topology attributes.

The extraction process (fig. 10) involves three steps:

1. **Parsing:** a vendor-specific parser reads the configuration and builds an internal syntax tree.
2. **Normalisation:** the syntax tree is mapped to plan topology attributes using vendor-specific rules (e.g., `ip address 10.0.1.1 255.255.255.252` becomes `ipv4_address=10.0.1.1/30`).
3. **Fragment emission:** the normalised attributes are packaged as a plan fragment with the appropriate node and endpoint identifiers.

Lossy Extraction

Vendor configuration extraction is inherently lossy. Some vendor-specific features (e.g., IOS `ip tcp adjust-mss`) have no corresponding plan topology attribute. The extraction process should log unrecognised configuration lines for manual review.

3.6 Security Segmentation Policy

Security policies define access control, segmentation, and encryption requirements. They typically originate from security teams rather than network engineers, and are expressed at a higher abstraction level.

Security Segmentation Policy

```
segmentation:
  zones:
    - name: trusted
      members: [leaf-01, leaf-02]
```

```

- name: dmz
  members: [leaf-03]
- name: untrusted
  members: [leaf-04]

policies:
- from: untrusted
  to: trusted
  action: deny

- from: trusted
  to: dmz
  action: permit
  acl:
    - permit tcp any any eq 443
    - permit tcp any any eq 80
    - deny ip any any

- from: dmz
  to: trusted
  action: permit
  acl:
    - permit tcp any any eq 3306
    - deny ip any any

```

The security policy compiler translates zone-based rules into plan topology attributes:

- Zone membership becomes `security_zone` attributes on nodes.
- Inter-zone policies become `acl_*` attributes on the endpoints connecting zones.
- Encryption requirements become `macsec_*` or `ipsec_*` attributes.

3.7 QoS Specifications

Quality of Service specifications define traffic classification, marking, queuing, and shaping policies. Like security policies, they often originate from a different team (capacity planning or service management) and are expressed in terms of service levels rather than device configurations.

QoS Specification

```

qos:
  traffic_classes:
    - name: voice
      dscp: ef
      priority: strict
      bandwidth_percent: 10

    - name: video
      dscp: af41
      bandwidth_percent: 30
      burst_percent: 10

    - name: business_critical
      dscp: af31
      bandwidth_percent: 25

    - name: best_effort
      dscp: 0
      bandwidth_percent: 35

  application_to:
    match:
      role: [spine, leaf]
    direction: egress

```

The QoS compiler maps these specifications to `qos_*` attributes on the matched endpoints, producing the classification, marking, and scheduling attributes required by the QoS functor $\mathcal{F}_{\text{QoS}}$.

Modality	Primary Output	Example Attributes	Abstraction
Topology Fragments	$V, \mathcal{E}, E_{\text{conn}}$	hostname, role, platform	Low
Design Templates	Complete fragments	All protocol attributes	Medium
Policy Intent	Attribute assignments	bgp_*, ospf_*, isis_*	High
Source of Truth	Inventory + IP	hostname, ipv4_address, vlan_id	Low
Vendor Extraction	Normalised config	All observed attributes	Low
Security Policy	ACL + segmentation	acl_*, sgt_*, macsec_*	High
QoS Specs	Traffic policy	qos_*, dscp_*	High
Compliance	Validation gates	Various (enforce/deny)	High

Figure 11. Summary of input modalities, their primary outputs, typical attributes produced, and abstraction level.

3.8 Compliance and Regulatory Constraints

Compliance constraints express mandatory requirements imposed by regulations, standards, or organisational policies. Unlike other input modalities, compliance constraints are typically expressed as *negative* rules (“must not”) or *invariant* rules (“must always”).

Compliance Constraints

```

compliance:
- rule: encrypt-management
  description: "All management traffic must use encryption"
  constraint:
    match:
      endpoint: management
    require:
      ssh_enabled: true
      telnet_enabled: false
      snmpv3_enabled: true
      snmpv2c_enabled: false

- rule: dual-homed-wan
  description: "Every site must have at least two WAN uplinks"
  constraint:
    match:
      role: wan_edge
    require_min_links:
      peer_role: wan_provider
      count: 2

```

Compliance constraints differ from other modalities in that they primarily act as *validation gates* rather than attribute producers. They are evaluated after composition to verify that the assembled plan satisfies all mandatory requirements. However, some compliance rules do produce attributes directly (e.g., forcing `ssh_enabled=true`).

3.9 Modality Summary

Figure 11 summarises the eight input modalities. The key observation is that they span a wide range of abstraction levels: topology fragments and vendor extractions operate at the individual device/interface level, while policies and compliance constraints operate at the group or network-wide level. The composition pipeline (section 4) must bridge these abstraction levels, compiling high-level intent into flat attributes and merging them with low-level specifics.

4 The Composition Pipeline

The composition pipeline transforms heterogeneous intent sources into a valid plan topology through six sequential stages. Each stage has well-defined inputs, outputs, and mathematical invariants.

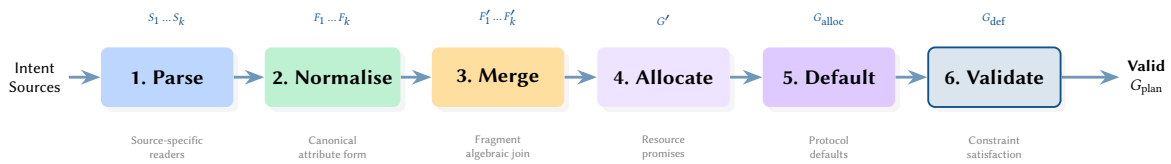


Figure 12. The six-stage composition pipeline. Raw intent sources are parsed into fragments, normalised, algebraically merged, allocated resources, populated with defaults, and finally validated against invariant constraints.

4.1 Architectural Framing: The Narrow Waist

Drawing on Shenker’s principle of a “narrow waist” architecture and Liskov’s principles of data abstraction, the pipeline acts as an abstraction layer that strictly decouples *Intent Expression* from *Network State Verification*.

In this architecture, the *Canonical Fragment* (F'_i) and the *Merged Graph* (G') form the narrow waist. Intent sources are treated as opaque abstract types; the pipeline only concerns itself with their behavioral contract (the attributes they assert) rather than their internal structure. By standardizing the algebraic representation of network intent, arbitrary intent sources (e.g., IETF Network Slice Intent per RFC 9315, RPSL policies per RFC 2622, YAML models, source-of-truth APIs) can map to arbitrary verification and deployment targets.

Ultimately, G_{plan} is not merely a data structure but an *Intermediate Representation (IR)* for network intent, bridging the gap between high-level policy and low-level device configuration.

4.2 Mathematical Foundations

To reason about the pipeline algebraically, we define a fragment F_i as a Property Graph:

$$F = (V, E, A, \Sigma)$$

where V represents the set of vertices (devices, endpoints), E is the set of edges (links, associations), A is the attribute mapping function, and Σ is the schema or alphabet of canonical attributes. This model conceptually extends the YANG Data Model for Network Topologies (RFC 8345) by overlaying abstract intent attributes onto the topological base.

When combining fragments, the merge operator $\oplus$ is defined such that the space of all possible graph fragments forms a *join-semilattice*. A join-semilattice is a partially ordered set that has a join (a least upper bound) for any nonempty finite subset. Consequently, the $\oplus$ operation strictly obeys three algebraic properties:

- **Commutativity:** $F_1 \oplus F_2 = F_2 \oplus F_1$
- **Associativity:** $(F_1 \oplus F_2) \oplus F_3 = F_1 \oplus (F_2 \oplus F_3)$
- **Idempotency:** $F_1 \oplus F_1 = F_1$

These properties guarantee that the final composed graph is entirely independent of the order in which the intent sources are parsed.

4.3 Stage 1: Parse

The parse stage reads each intent source S_i and produces a raw plan fragment F_i . Each source type has a dedicated parser:

- **YAML/JSON parser:** reads topology fragments, policy intent files, and templates.
- **Template expander:** evaluates parameterised templates into concrete fragments.
- **SoT adapter:** queries source-of-truth APIs and maps responses to fragments.
- **Vendor config parser:** parses CLI configurations via platform-specific grammars.
- **Policy compiler:** evaluates high-level policy intent (e.g., isolating tenants) into attribute assignments.

Design Rule 7: Parse Independence

Each parser operates on exactly one intent source and produces exactly one plan fragment. Parsers have no side effects and no access to other fragments. This ensures that parse failures are isolated and that sources can be added or removed without affecting other parsers.

4.4 Stage 2: Normalise

The normalise stage transforms each raw fragment F_i into a *canonical* fragment F'_i . Crucially, this stage is strictly limited to syntax coercion; no semantic inferences (such as inferring OSPF costs from speeds) occur here.

1. **Key canonicalisation:** attribute keys are normalised to the naming grammar (snake_case, protocol prefix, no vendor-specific abbreviations).

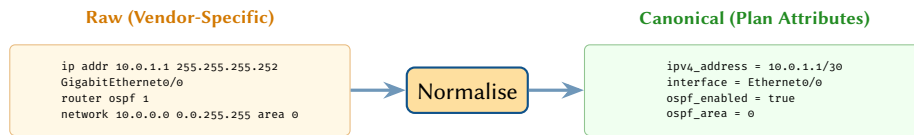


Figure 13. Normalisation transforms vendor-specific representations into canonical plan topology attributes.

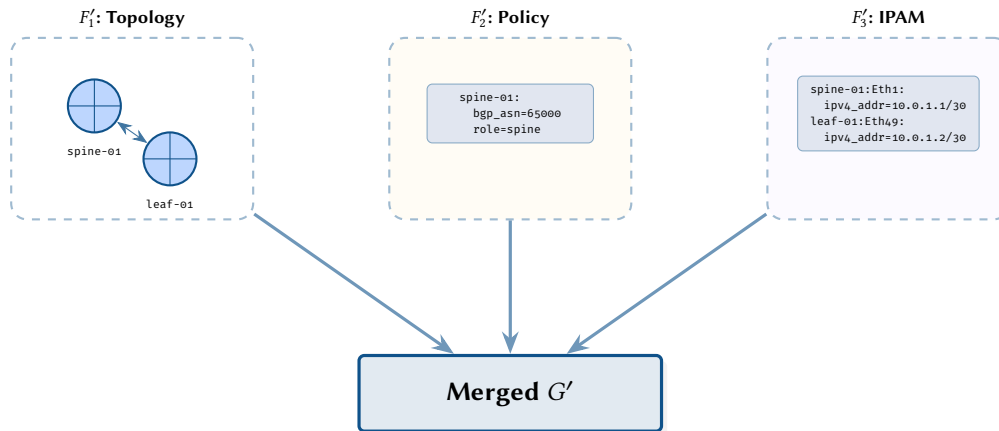


Figure 14. Fragment merge. Three fragments—topology (structure), policy (BGP attributes), and IPAM (addressing)—are merged into a single graph. The merged graph inherits all nodes, edges, and attributes from the contributing fragments.

2. **Value type coercion:** values are coerced to canonical types (e.g., subnet masks to CIDR, booleans to native).
3. **Identifier resolution:** device and interface names are resolved to canonical identifiers (e.g., `GigabitEthernet0/0` → `Ethernet0/0`).
4. **Resource Promises (Thunks):** Dynamic resource requests (e.g., `subnet: pool_a`) are parsed as unallocated variable promises. These “thunks” are retained for resolution in the Allocation stage.

4.5 Stage 3: Merge

The merge stage combines all normalised fragments $\{F'_1, \dots, F'_k\}$ into a single graph G' using a merge operator $\oplus$. Drawing inspiration from compositional network programming languages like Propane and Pyretic, the merge operator ensures that conflicting attributes are safely detected.

1. **Graph union:** the node sets, endpoint sets, and edge sets are unioned.
2. **Attribute overlay:** attribute maps from contributing fragments are overlaid. Conflicts trigger the deterministic resolution strategy.
3. **Cross-fragment edges:** connectivity edges that span fragments are created based on matching endpoints.

4.5.1 Conflict Resolution

When two fragments assign different values to the same attribute on the same entity, a conflict exists. To maintain algebraic commutativity, the resolution must be independent of processing order. Recency is explicitly rejected as an anti-pattern.

1. **Explicit priority:** each intent source has a configured priority. Higher-priority sources win (e.g., `security` > `policy` > `template`).
2. **Specificity:** a source that targets a specific node overrides one that targets a group.
3. **Rejection:** if priority and specificity are tied, the pipeline **must reject** the composition with a conflict error. This guarantees determinism.

4.6 Stage 4: Allocate

High-level policy intent frequently involves dynamic resource allocation (e.g., assigning a subnet from a pool, allocating an ASN, or dispensing VLAN IDs). The allocate stage transforms resource promises (“thunks”) left by the Normalise stage into concrete, static graph attributes.

Definition 2 (Resource Promise (Thunk)). A resource promise is a deferred allocation expression $\theta = (\text{pool}, \text{constraint}, \text{scope})$ where:

- *pool* $\in \mathcal{P}$ identifies the resource pool (e.g., an IP prefix pool, an ASN range, a VLAN registry).
- *constraint* specifies sizing or type requirements (e.g., `/30` for a point-to-point subnet, `/32` for a loopback).

Insight:
Normalisation is where
vendor diversity is absorbed.
After this stage, all
fragments speak the same
“language” of plan attributes.

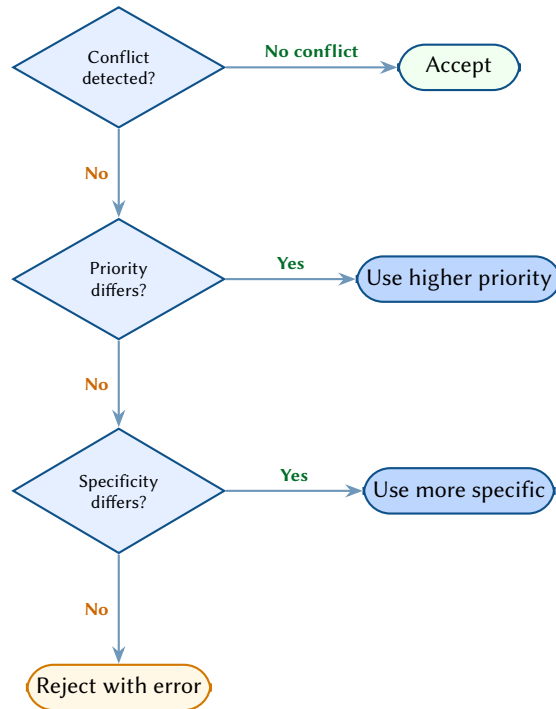


Figure 15. Conflict resolution decision tree. The pipeline resolves conflicts strictly by priority and specificity. Tied conflicts are rejected to maintain algebraic determinism.

- *scope determines the uniqueness domain: per-node, per-link, per-site, or global.*

The allocator resolves thinks by traversing the merged graph G' in a deterministic order (topological sort by node identifier) and replacing each promise with a concrete value drawn from the corresponding pool.

Design Rule 8: Allocation Determinism

Given the same merged graph G' and the same pool state, the allocator must produce identical assignments on every invocation. This is achieved by fixing the traversal order and using first-fit-decreasing allocation within each pool.

Design Rule 9: Global Uniqueness

No two entities in G_{alloc} may hold the same allocated value from the same pool within the same uniqueness scope. The allocator maintains a consumed-set per pool and rejects any allocation that would produce a duplicate.

4.6.1 Allocation Categories

The allocator handles four categories of resource:

1. **IP addressing:** point-to-point subnets (/30 or /31), loopback addresses (/32), VXLAN VTEPs, and anycast gateway IPs. Pool hierarchy supports nested super-pools and per-site sub-pools.
2. **Autonomous system numbers:** ASN allocation from a defined range (e.g., 65001–65534 for eBGP leaves). The allocator respects contiguity constraints when sites require sequential ASN blocks.
3. **VLAN and VNI identifiers:** VLAN IDs from site-scoped pools, VNI assignments from a global L2VNI/L3VNI range. Uniqueness is enforced within the broadcast domain for VLANs and globally for VNIs.
4. **Protocol identifiers:** OSPF area IDs, route distinguishers, route targets, MPLS label ranges, and STP bridge priorities. Each has its own uniqueness scope and allocation strategy.

4.6.2 Pool Hierarchy and Sub-Allocation

Resource pools support hierarchical sub-allocation for multi-site deployments. A super-pool (e.g., 10.0.0.0/8) is partitioned into per-site sub-pools, each of which is further partitioned by usage (P2P links, loopbacks, service VIPs):

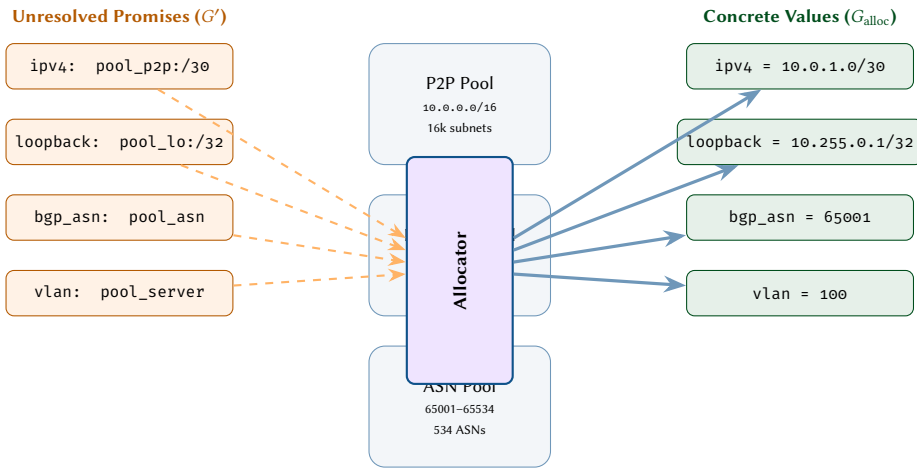


Figure 16. Thunk resolution in the Allocate stage. Resource promises (left) are resolved against pools to produce concrete attribute values (right). The allocator guarantees determinism and global uniqueness across the plan.

Hierarchical IP Pool

```
pools:
  fabric_addressing:
    type: ipv4
    prefix: 10.0.0.0/8
    sub_pools:
      - name: dc1_p2p
        prefix: 10.1.0.0/16
        allocation_size: /30
      - name: dc1_loopbacks
        prefix: 10.1.255.0/24
        allocation_size: /32
      - name: dc2_p2p
        prefix: 10.2.0.0/16
        allocation_size: /30
```

The allocator verifies that sub-pool ranges do not overlap and that the sum of sub-pool sizes does not exceed the super-pool capacity.

4.6.3 Allocation Failures

When a pool is exhausted or a constraint cannot be satisfied, the allocator produces a structured error with the pool name, the requesting entity, the current pool utilisation, and the thunk expression that failed. These errors are propagated to the validation report with full provenance.

4.7 Stage 5: Default

After allocation, the graph G_{alloc} may still have missing attributes required by downstream functors. The default stage fills these gaps using a layered strategy. Unlike early normalization stages, this phase applies semantic network inferences.

1. **Protocol defaults:** standard values from protocol specifications (e.g., OSPF cost derived from interface bandwidth, BGP hold time).
2. **Platform defaults:** values specific to device platforms (e.g., default MTUs).
3. **Role-based defaults:** values inferred from device roles (e.g., RR clients vs Route Reflectors).
4. **Organisational defaults:** site-wide conventions.

Design Rule 10: Explicit Over Default

A value explicitly set by any intent source, or dynamically assigned during allocation, always takes precedence over a static default.

4.8 Stage 6: Validate

The final stage frames plan validation as a formal constraint satisfaction problem where $G_{\text{def}} \models \Phi$ (the graph satisfies the set of invariants Φ). Unlike systems like Batfish which validate low-level vendor configurations, this stage validates the *abstract intent graph* before configurations are generated.

1. **Structural validation:** graph-level invariants (bidirectional connectivity, no orphan nodes).

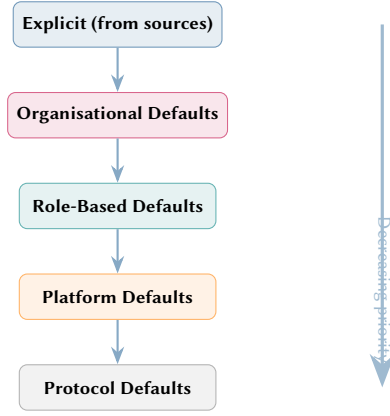


Figure 17. Default value resolution order. Explicitly-set values from intent sources always win. If no explicit value exists, defaults are tried from organisational through to protocol-level, with earlier layers taking precedence.

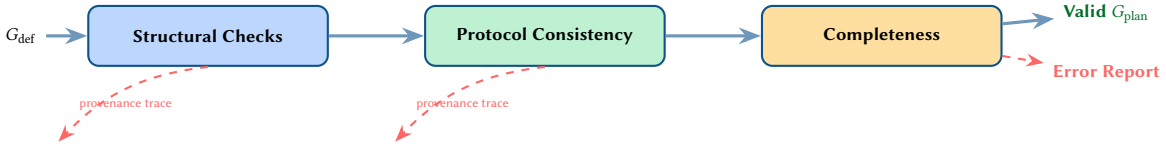


Figure 18. Three-phase validation. Each phase produces either a pass (forwarding to the next phase) or a structured error report with provenance traces back to the responsible intent sources.

2. **Protocol consistency:** inter-device agreement (OSPF timer match, BGP peer reciprocity, IP uniqueness).
3. **Completeness:** ensuring every active protocol instance has requisite attributes.

If validation succeeds, the output is a *valid* G_{plan} , ready for consumption by the functorial overlay mappings. If validation fails, the pipeline produces a structured error report identifying the failing rule, the affected entities, and—via provenance tracing—the intent sources responsible.

4.9 Case Study: Tracing Intent through the Pipeline

To make the abstract pipeline concrete, consider a simple operator intent: “Isolate Guest Wi-Fi traffic across the campus.” We trace how this high-level directive is refined into the unified G_{plan} :

1. **Parse:** A high-level YAML file expressing intent: `isolate for target: guest-wifi` is ingested by the Policy Compiler parser.
2. **Normalise:** The intent is syntactically coerced into a canonical fragment F'_i . The policy dictates a VRF assignment. Here, we encounter a Thunk: the fragment records `vrf = guest` and a resource promise `vlan = {pool: guest_vlans}`.
3. **Merge:** F'_i is combined via $\oplus$ with the base physical topology graph G . The guest VRF properties are overlaid onto the campus access switches.
4. **Allocate:** The allocation traversal observes the promise `{pool: guest_vlans}`. It queries the pool, guarantees global contiguity, and permanently writes `vlan = 100` to the graph.
5. **Default:** The pipeline infers semantic network properties not specified in the intent. For the newly established VRF, it applies the standard organizational BGP timers and platform-specific MTU sizes.
6. **Validate:** The constraint solver checks invariants. It confirms that `vlan = 100` is uniformly assigned across all required access links and that the VRF route targets adhere to structural rules. The result is a valid G_{plan} ready for downstream extraction.

4.10 End-to-End Attribute Lifecycle

To illustrate how a single attribute flows through all six stages, fig. 19 traces the `ipv4_address` attribute on a leaf switch’s uplink interface from initial intent to validated plan.

4.11 Pipeline Properties

Theorem 1 (Pipeline Determinism). *Given the same set of intent sources and the same conflict resolution configuration, the composition pipeline always produces the same G_{plan} (or the same error report).*

Proof. Each stage is a pure function. Parse is deterministic per source; normalise applies fixed rules; merge uses a commutative, associative operator ($\oplus$) with deterministic conflict rejection; allocation uses deterministic allocation algorithms; defaults apply fixed priority layers; and validate checks fixed constraints. $\square$

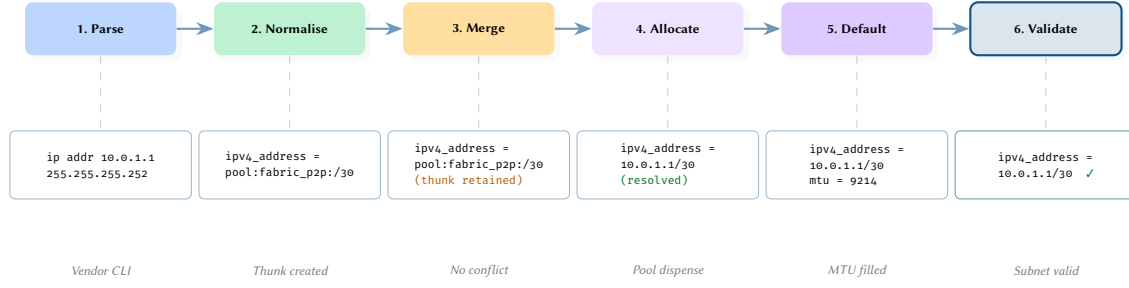


Figure 19. End-to-end attribute lifecycle. A single attribute (`ipv4_address`) is traced through all six pipeline stages: from raw vendor CLI, through thunk creation, merge, pool allocation, default enrichment, and final validation.

Theorem 2 (Incremental Re-Composition). *If a single intent source S_j changes while all others remain unchanged, the merge stage can reuse cached fragments from unchanged sources.*

Proof. Since $\oplus$ is commutative and associative, incremental updates are algebraically straightforward. We define $\ominus$ as a relative complement (retraction) operation that safely removes the specific attributes asserted by a fragment. The updated graph can be mathematically derived as:

$$G_{\text{new}} = G_{\text{old}} \ominus F_{\text{old}} \oplus F_{\text{new}}$$

Because the conflict resolution model forbids recency bias, this guarantees $O(1)$ re-composition complexity relative to the total number of fragments. $\square$

5 Fragment Algebra

This section defines the algebraic structure of plan fragment composition. We establish three operators—union, overlay, and override—and characterise their algebraic properties.

5.1 Plan Fragments as Partial Graphs

Recall from eq. (1) that a plan fragment F is a partial plan topology. We now make this precise:

Definition 3 (Plan Fragment). A plan fragment is a tuple $F = (V_F, \mathcal{E}_F, E_{\text{conn}}^F, E_{\text{own}}^F, \pi_V^F, \pi_{\mathcal{E}}^F)$ where:

- $V_F \subseteq \mathcal{U}_V$ is a finite set of node identifiers drawn from a universal node set.
- $\mathcal{E}_F \subseteq \mathcal{U}_{\mathcal{E}}$ is a finite set of endpoint identifiers.
- $E_{\text{conn}}^F \subseteq \mathcal{E}_F \times \mathcal{E}_F$ is the connectivity edge set.
- $E_{\text{own}}^F \subseteq V_F \times \mathcal{E}_F$ is the ownership edge set.
- $\pi_V^F : V_F \rightarrow \mathbb{A}$ is a partial attribute map on nodes.
- $\pi_{\mathcal{E}}^F : \mathcal{E}_F \rightarrow \mathbb{A}$ is a partial attribute map on endpoints.

The notation $f \rightarrow A$ denotes a partial function: $\pi_V^F(v)$ returns a partial key-value map, where some keys may be undefined.

5.2 The Union Operator ($\cup$)

The simplest composition operator is graph union, which combines fragments that describe disjoint parts of the network.

Definition 4 (Fragment Union). Given fragments F_1 and F_2 , their union $F_1 \cup F_2$ is defined component-wise:

$$V_{F_1 \cup F_2} = V_{F_1} \cup V_{F_2} \quad (3)$$

$$\mathcal{E}_{F_1 \cup F_2} = \mathcal{E}_{F_1} \cup \mathcal{E}_{F_2} \quad (4)$$

$$E_{\text{conn}}^{F_1 \cup F_2} = E_{\text{conn}}^{F_1} \cup E_{\text{conn}}^{F_2} \quad (5)$$

$$E_{\text{own}}^{F_1 \cup F_2} = E_{\text{own}}^{F_1} \cup E_{\text{own}}^{F_2} \quad (6)$$

For attribute maps, if a node $v \in V_{F_1} \cap V_{F_2}$ and both fragments define the same key k :

$$\pi_V^{F_1 \cup F_2}(v)(k) = \begin{cases} \pi_V^{F_1}(v)(k) & \text{if } \pi_V^{F_1}(v)(k) = \pi_V^{F_2}(v)(k) \\ \perp_{\text{conflict}} & \text{otherwise} \end{cases}$$

That is, union requires agreement on overlapping attributes. Conflicting values produce an error.

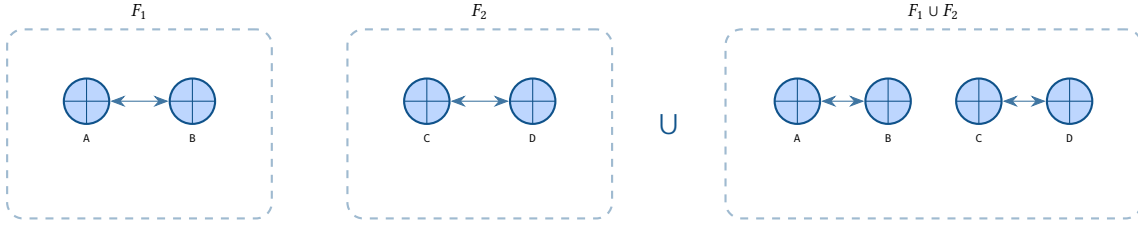


Figure 20. Fragment union on disjoint fragments. The result contains all nodes and edges from both fragments.

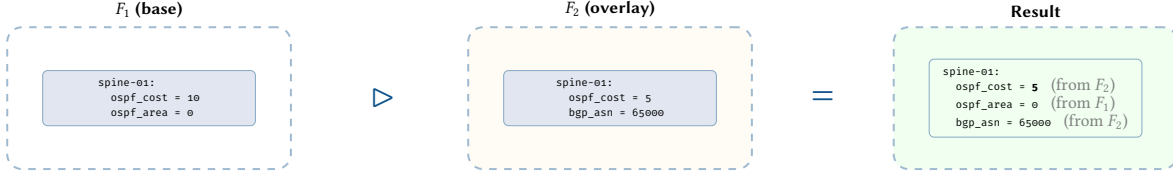


Figure 21. Fragment overlay. F_2 's attributes take precedence over F_1 's on conflict (`ospf_cost`), while non-conflicting attributes from both fragments are preserved.

Proposition 1 (Union Properties). *Fragment union is:*

1. **Commutative:** $F_1 \cup F_2 = F_2 \cup F_1$.
2. **Associative:** $(F_1 \cup F_2) \cup F_3 = F_1 \cup (F_2 \cup F_3)$.
3. **Idempotent:** $F \cup F = F$.

These properties hold because set union on each component is commutative, associative, and idempotent, and the attribute agreement check is symmetric.

5.3 The Overlay Operator ($\triangleright$)

The overlay operator extends union by allowing one fragment to supply attributes for entities defined in another fragment, without requiring agreement.

Definition 5 (Fragment Overlay). *Given fragments F_1 and F_2 , the overlay $F_1 \triangleright F_2$ (read “ F_1 overlaid with F_2 ”) combines their graph structure via union and resolves attribute conflicts by preferring F_2 :*

$$\pi_{V}^{F_1 \triangleright F_2}(v)(k) = \begin{cases} \pi_V^{F_2}(v)(k) & \text{if defined in } F_2 \\ \pi_V^{F_1}(v)(k) & \text{otherwise} \end{cases}$$

Overlay is the natural operator for layering intent: a base topology fragment provides defaults, and a policy fragment overlays specific attribute values.

Proposition 2 (Overlay Properties). *Fragment overlay is:*

1. **Associative:** $(F_1 \triangleright F_2) \triangleright F_3 = F_1 \triangleright (F_2 \triangleright F_3)$.
2. **Not commutative:** in general, $F_1 \triangleright F_2 \neq F_2 \triangleright F_1$.
3. **Idempotent:** $F \triangleright F = F$.

5.4 The Override Operator ($\gg$)

The override operator is a stricter form of overlay that *replaces* all attributes on matching entities, rather than merging them key-by-key.

Definition 6 (Fragment Override). *Given fragments F_1 and F_2 , the override $F_1 \gg F_2$ replaces the entire attribute map for any entity present in both fragments:*

$$\pi_V^{F_1 \gg F_2}(v) = \begin{cases} \pi_V^{F_2}(v) & \text{if } v \in V_{F_2} \\ \pi_V^{F_1}(v) & \text{otherwise} \end{cases}$$

Override is used when a source provides a *complete* replacement for an entity's configuration—for example, when a vendor config extraction should entirely replace the template-generated attributes for a specific device.

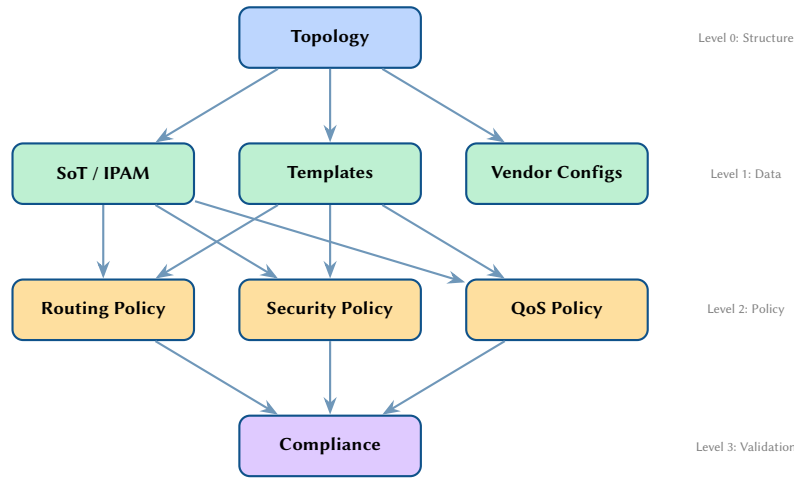


Figure 22. The fragment DAG. Fragments at each level depend on fragments from earlier levels. Fragments at the same level can be composed in parallel. The DAG determines the evaluation order for the merge stage.

Override Destroys Attributes

Unlike overlay, which preserves non-conflicting attributes from the base fragment, override discards *all* base attributes for affected entities. Use override only when the overriding source provides a complete attribute set.

5.5 Composition Order and the Fragment DAG

In practice, fragments are not composed in arbitrary order. The composition pipeline defines a *fragment DAG* that specifies dependencies:

Design Rule 11: DAG-Ordered Composition

Fragments are composed in topological order of the fragment DAG. At each level, fragments are composed using the overlay operator ($\triangleright$) with left-to-right priority. A fragment at level n may reference attributes produced by fragments at levels $0, \dots, n-1$, but not from other fragments at level n .

This DAG structure mirrors the natural dependency order of network design: you must define the topology before you can assign IP addresses, and you must know the addressing before you can write routing policies.

5.6 Algebraic Laws

The following laws govern the interaction of the three operators:

Theorem 3 (Absorption). For any fragment F : $F \cup F = F \triangleright F = F \gg F = F$.

Theorem 4 (Union-Overlay Compatibility). If fragments F_1 and F_2 have no attribute conflicts (i.e., they agree on all overlapping keys), then: $F_1 \cup F_2 = F_1 \triangleright F_2 = F_2 \triangleright F_1$.

Proof. When there are no conflicts, the overlay preference rule is never triggered. The attribute maps are identical regardless of overlay direction, reducing overlay to union. $\square$

Theorem 5 (Override Subsumes Overlay). For any fragments F_1, F_2 : if every entity in F_2 has a complete attribute map (i.e., defines all attributes required by the schema), then $F_1 \triangleright F_2 = F_1 \gg F_2$.

These algebraic properties are important for reasoning about composition correctness and for optimising the merge stage (e.g., reordering independent unions for parallelism).

5.7 Categorical Properties of Composition

To further formalise the composition engine, we define the **Category of Plan Fragments** Frag .

- **Objects:** Well-formed plan fragments F .
- **Morphisms:** A morphism $m : F_1 \rightarrow F_2$ exists if $F_1 \sqsubseteq F_2$, representing a process of structural or informational refinement.

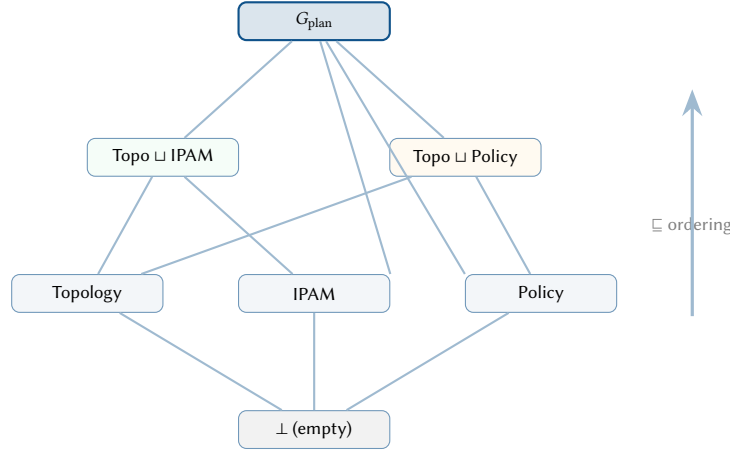


Figure 23. Join-semilattice of plan fragments. Each fragment represents a partial view of the plan; the join ($\sqcup$) combines them monotonically. The complete G_{plan} is the top element after all fragments are joined.

The extraction process $\mathcal{F}$ is a **functor** $\mathcal{F} : \text{Frag} \rightarrow \text{Overlay}$. This functorial mapping must satisfy the following properties:

1. **Structure Preservation:** If $F_1 \rightarrow F_2$ is a morphism in **Frag** (i.e., F_2 refines F_1), then there exists a morphism $\mathcal{F}(F_1) \rightarrow \mathcal{F}(F_2)$ in **Overlay**. This ensures that adding more intent to the plan never destroys existing overlay reachability.
2. **Compositionality:** $\mathcal{F}(F_1 \oplus F_2) \cong \mathcal{F}(F_1) \oplus \mathcal{F}(F_2)$. The extraction of a merged plan is isomorphic to the merge of individual extractions. This property is crucial for distributed compilation and incremental updates.

By grounding the "How We Built It" in these categorical laws, we ensure that the NTE engine is not merely a collection of heuristics, but a rigorous implementation of algebraic graph theory.

5.8 Join-Semilattice Structure

The algebraic properties of the union operator ($\cup$)—commutativity, associativity, and idempotence—establish that the space of plan fragments forms a *join-semilattice* $(\mathcal{F}, \sqsubseteq, \sqcup)$, where:

- $\mathcal{F}$ is the set of all well-formed plan fragments.
- $F_1 \sqsubseteq F_2$ iff $F_1 \cup F_2 = F_2$ (i.e., F_1 's information is subsumed by F_2).
- $F_1 \sqcup F_2 = F_1 \cup F_2$ is the join (least upper bound) of two fragments.

The semilattice structure (fig. 23) guarantees several important properties:

1. **Monotonicity:** composing additional fragments can only *add* information, never remove it (unless using the override operator, which operates outside the semilattice).
2. **Confluence:** the final result is independent of composition order, since the join is commutative and associative.
3. **Incremental growth:** fragments can be added incrementally; each join step moves strictly upward in the lattice.

6 Policy Compilation

Policy intent is the highest-abstraction input modality: operators describe *what* the network should do, and the policy compiler translates that into the flat attribute assignments that the plan topology requires. This translation is strictly governed by a set of **Design Rules**, which consists of *Primitives* (the core building blocks or objects of intent) and *Assertions* (the structural guarantees required by the architecture). This section formalises the compilation process and how Structural Composition intent bridges into our rigorous mathematical model.

6.1 Design Rules: Primitives and Assertions

To effectively translate topological specification into rigorous graph mathematics, the policy compiler strictly adheres to a predefined set of **Design Rules**. These rules prevent the generation of "spaghetti" intent and ensure that every compiled attribute serves a defined architectural purpose.

Design Rules are divided into two categories:

1. **Primitives (The Nouns):** These are the foundational objects of intent that can be manipulated on The Structural Baseline or via the DSL. Examples include a VRF, a VLAN, a BGP_Peer_Group, or a

Insight:
The semilattice perspective explains why the merge operator $\oplus$ in the composition pipeline (section 4.5) is well-behaved: it is simply the join operation on a semilattice of partial network descriptions.

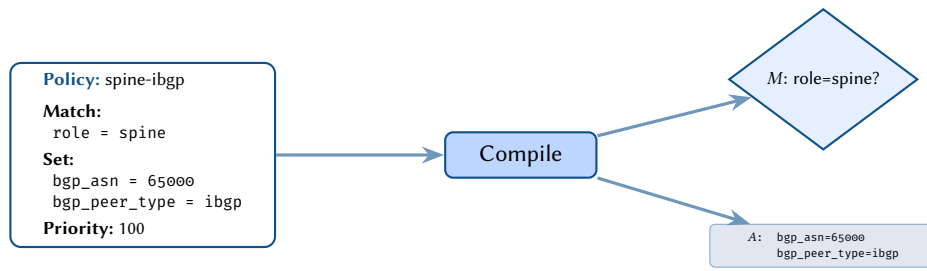


Figure 24. Policy decomposition. A policy statement is compiled into a match predicate M and an attribute assignment A .

Security_Zone. The compiler is only allowed to generate plan attributes that correspond to these registered Primitives.

2. **Assertions (The Verbs/Constraints):** These are the structural guarantees required by the architecture. For example, an Assertion might state: "A VRF Primitive must not overlap IP space with another VRF unless explicitly joined by a Route Leaking Primitive." The compiler embeds these assertions as validation gates during the merge process.

By framing the compilation through Primitives and Assertions, operators are forced to think in terms of standardized building blocks rather than bespoke CLI commands.

6.2 Policy Language Concepts

A policy P consists of three components:

Definition 7 (Policy). A policy is a triple $P = (M, A, \rho)$ where:

- $M : V \cup \mathcal{E} \rightarrow \{true, false\}$ is a match predicate that selects which entities the policy applies to.
- $A : K \rightarrow V$ is an attribute assignment—a partial function mapping attribute keys to values.
- $\rho \in \mathbb{N}$ is the priority of the policy, used for conflict resolution when multiple policies match the same entity.

6.3 Match Predicates

Match predicates operate on the current state of the plan graph (after topology and data fragments have been merged). They support the following operations:

- **Attribute equality:** `role = spine`.
- **Attribute membership:** `role in [spine, border_leaf]`.
- **Attribute pattern:** `hostname matches "dc1-spine-*"`.
- **Topology predicates:** `connected_to(role=spine)` matches endpoints that are connected to a spine.
- **Boolean combinators:** `and`, `or`, `not` for composing predicates.

Compound Match Predicate

```
match:
  and:
    - role: leaf
    - connected_to:
        role: spine
    - not:
        hostname: "leaf-reserved-*
```

This matches all leaf nodes that are connected to at least one spine, excluding nodes whose hostname matches the reserved pattern.

6.4 Attribute Generators

Not all attribute assignments are static values. The policy language supports *attribute generators* that compute values dynamically:

- **Auto-increment:** `bgp_asn: "auto:65001+index"` assigns ASNs starting from 65001, incrementing per matched node.
- **Derived values:** `router_id: "from:loopback0.ipv4_address"` copies the value from another attribute.
- **Computed values:** `ospf_cost: "compute:ref_bw / phy_speed"` evaluates an expression using other attributes.

Insight: Topology-aware predicates like `connected_to` are what make the policy language significantly more expressive than flat attribute selectors. They allow policies to reason about network structure.

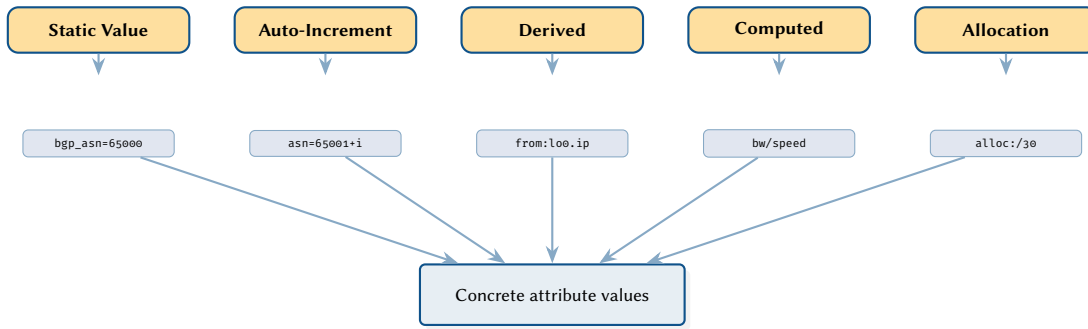


Figure 25. Attribute generator types. Each generator produces concrete attribute values from different computation strategies.

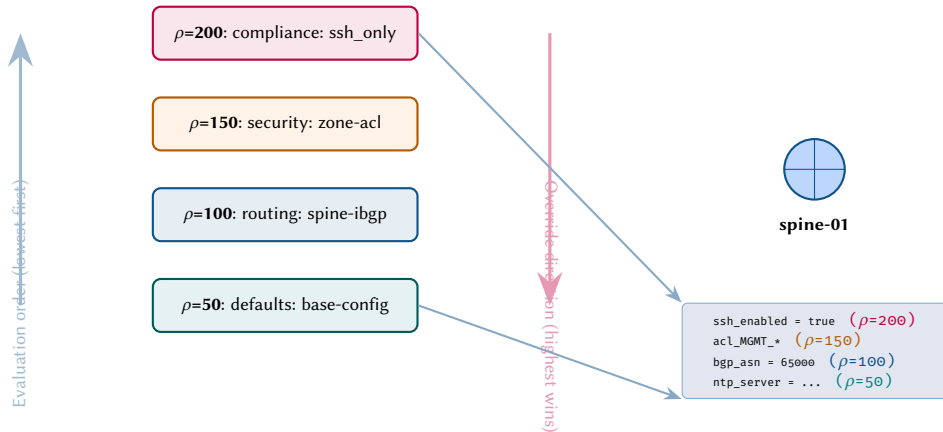


Figure 26. Policy evaluation order. Lower-priority policies are evaluated first (providing defaults); higher-priority policies overlay on top. Each attribute in the result is traceable to the policy that produced it.

- **Allocation:** ipv4_address: "allocate:10.0.0.0/16:p2p:/30" allocates a prefix from a pool.

6.5 Policy Evaluation Order

When multiple policies match the same entity, their interactions must be well-defined. The policy compiler evaluates policies in priority order (highest ρ first) and uses the overlay operator for combination:

$$F_{\text{policy}} = P_1 \triangleright P_2 \triangleright \dots \triangleright P_n \quad \text{where} \quad \rho(P_1) \geq \rho(P_2) \geq \dots \geq \rho(P_n) \quad (7)$$

This means lower-priority policies serve as defaults that higher-priority policies can override.

6.6 Policy Composition Patterns

Several common patterns emerge from policy composition:

6.6.1 Layered Defaults

A common pattern is to define a hierarchy of increasingly specific policies:

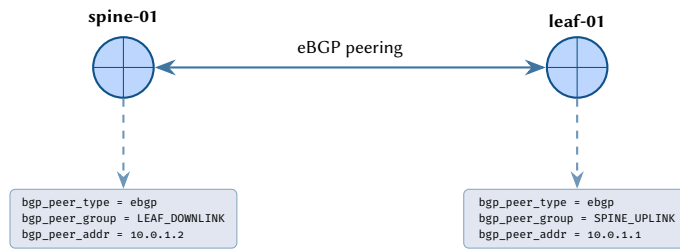
Layered Policy Defaults

```

policies:
  # Layer 1: global defaults (lowest priority)
  - name: global-defaults
    priority: 10
    match: {all: true}
    set:
      ospf_hello_interval: 10
      ospf_dead_interval: 40
      bgp_hold_time: 180

  # Layer 2: role-specific
  - name: spine-config
    priority: 50
    match: {role: spine}
    set:

```



Cross-device policy applies attributes to both endpoints of matching links

Figure 27. Cross-device policy compilation. The policy matches links between spines and leaves and applies complementary attributes to both endpoints.

```

    bgp_rr_client: false
    ospf_cost: 10

    # Layer 3: site-specific
    - name: dc1-spines
      priority: 80
      match:
        and:
          - role: spine
          - site: dc1
      set:
        bgp_asn: 65001
        ospf_area: 0

```

6.6.2 Cross-Device Policies

Some policies express relationships *between* devices rather than attributes on a single device. These generate attributes on endpoints rather than nodes:

Cross-Device Policy – BGP Peering

```

policies:
  - name: spine-leaf-ebgp
    type: peering
    match_local: {role: leaf}
    match_peer: {role: spine}
    set_local:
      bgp_peer_type: ebgp
      bgp_peer_group: SPINE_UPLINK
    set_peer:
      bgp_peer_type: ebgp
      bgp_peer_group: LEAF_DOWNLINK

```

Cross-device policies are evaluated by iterating over all connectivity edges and checking the match predicates on both endpoints. When a match is found, attributes are applied to both sides of the link.

6.6.3 Conditional Policies

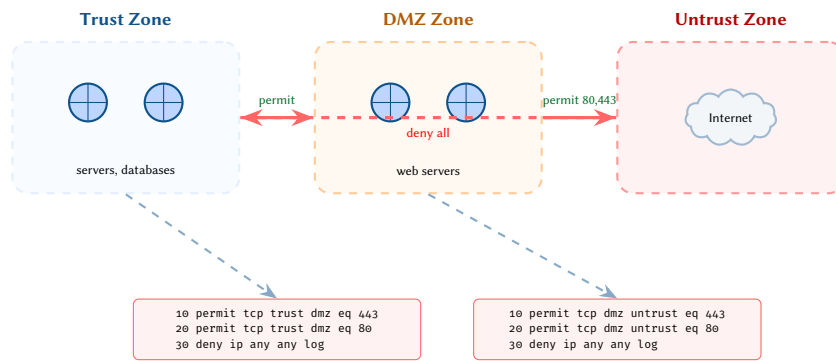
Policies can be conditioned on the *result* of earlier policy evaluation, enabling cascading configuration:

Conditional Policy

```

policies:
  - name: bfd-for-ebgp
    priority: 120
    match:
      bgp_peer_type: ebgp      # attribute set by earlier policy
    set:
      bfd_enabled: true
      bfd_interval: 300

```



Policy compiler generates per-interface ACLs from zone-pair rules

Figure 28. Zone-based firewall compilation. High-level zone-pair rules (top) are compiled into per-interface ACLs (bottom) on devices at zone boundaries.

```
bfd_multiplier: 3
```

This policy enables BFD on all eBGP sessions—but it matches on `bgp_peer_type`, which is itself set by an earlier routing policy. The fragment DAG (fig. 22) ensures that the routing policy is evaluated before this one.

6.7 Firewall and ACL Policy Compilation

Firewall policies present unique composition challenges because they involve *ordered rule sets* where the sequence of rules matters, and because they often span multiple devices (inter-zone rules require coordinated ACLs on ingress and egress points).

6.7.1 Zone-Based Firewall Model

The policy compiler maps high-level security zone specifications into per-device ACL entries. A zone definition produces three types of plan attributes:

1. **Zone membership:** assigns `security_zone` to each node or interface.
2. **Inter-zone rules:** generates ACL entries on interfaces at zone boundaries.
3. **Intra-zone policy:** determines whether same-zone traffic is implicitly permitted or requires explicit rules.

Zone-Based Firewall Policy

```
security:
  zones:
    - name: trust
      members: [leaf-01, leaf-02]
    - name: dmz
      members: [leaf-03]
    - name: untrust
      members: [border-01]
  zone_pairs:
    - from: trust
      to: dmz
      rules:
        - permit: {protocol: tcp, port: [80, 443]}
    - from: dmz
      to: untrust
      rules:
        - permit: {protocol: tcp, port: [80, 443]}
    - from: untrust
      to: trust
      action: deny_all
  default_action: deny
```

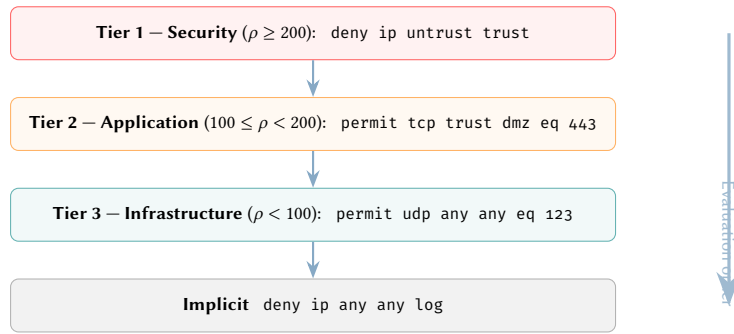


Figure 29. ACL composition tiers. Security rules are placed first (highest priority), followed by application and infrastructure rules. An implicit deny-all terminates every composed ACL.

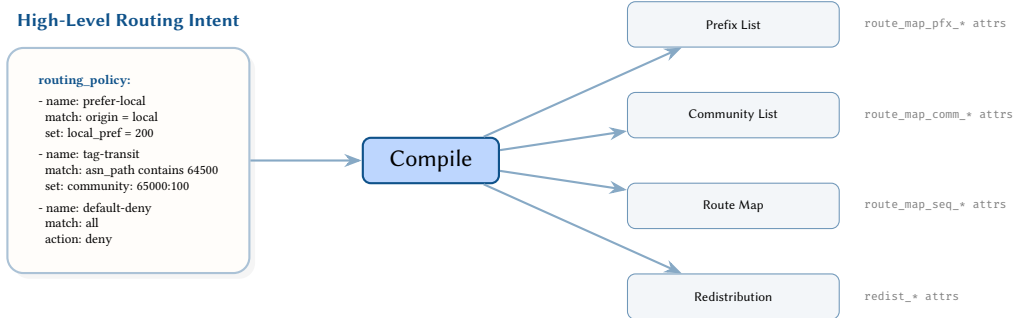


Figure 30. Route policy compilation. High-level routing intent is decomposed into structured plan attributes for prefix lists, community lists, route maps, and redistribution rules.

6.7.2 ACL Composition and Ordering

When multiple policy sources contribute ACL rules to the same interface, the composition pipeline must produce a single, correctly ordered rule set. This is fundamentally different from flat attribute merge: ACLs are *ordered sequences*, not sets.

Design Rule 12: ACL Composition Order

ACL rules are composed in three tiers, each tier taking precedence over the next:

1. **Security policies:** zone-pair deny rules, blacklists, quarantine rules ($\rho \geq 200$).
2. **Application policies:** service-specific permit rules, traffic classification ($100 \leq \rho < 200$).
3. **Infrastructure policies:** management access, NTP, DNS, SNMP permit rules ($\rho < 100$).

Within each tier, rules are ordered by specificity (most specific match first). An implicit deny-all terminates every ACL.

6.7.3 Stateful Firewall Rule Expansion

When the policy compiler encounters a stateful allow rule (e.g., “permit established TCP from DMZ to Internet”), it generates reciprocal entries:

1. A forward rule on the egress interface permitting the initiating flow.
2. A return rule on the ingress interface permitting established return traffic (using established or reflexive ACL markers).
3. Provenance annotations on both rules tracing back to the single zone-pair policy that generated them.

This is a cross-device policy pattern (section 6): the single zone-pair intent statement expands into correlated ACL entries on multiple devices.

6.8 Route Policy Compilation

Route policies (route-maps, prefix-lists, community-lists) are compiled from high-level routing intent into structured plan attributes.

6.8.1 Route-Map Sequence Generation

The policy compiler converts high-level routing preferences into ordered route-map sequences stored as plan attributes:

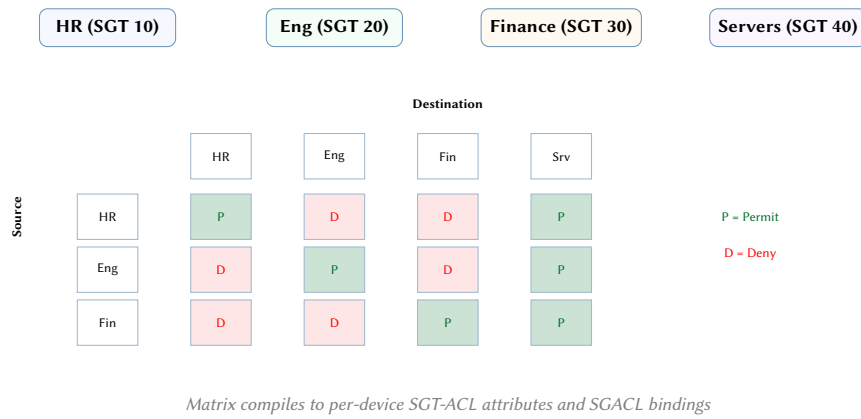


Figure 31. SGT-based micro-segmentation. A source-destination access matrix (centre) defines group-level permissions. The policy compiler generates per-device SGACL bindings and enforcement attributes.

Route-Map Compilation

```
# High-level intent
routing_policy:
- name: ebgp-in
  apply_to:
    match: {role: leaf, bgp_peer_group: SPINE_UPLINK}
  sequences:
    - match: {prefix_list: DC_LOCAL}
      set: {local_pref: 200}
    - match: {community: "64500:*"}
      set: {local_pref: 50}
    - action: permit # default accept

# Compiled to plan attributes on each matching endpoint:
# route_map_in_name = "EBGP-IN"
# route_map_in_seq_10_match = "prefix-list DC_LOCAL"
# route_map_in_seq_10_set = "local-preference 200"
# route_map_in_seq_20_match = "community 64500:*"
# route_map_in_seq_20_set = "local-preference 50"
# route_map_in_seq_30 = "permit"
```

6.8.2 Redistribution Policy

When multiple routing protocols coexist, redistribution policies control which routes leak between protocol domains. The policy compiler handles mutual redistribution loops by generating route-tag-based loop prevention:

1. Routes redistributed from OSPF into BGP are tagged with a protocol-origin tag.
2. The reciprocal redistribution (BGP into OSPF) filters routes carrying the OSPF-origin tag, preventing feedback loops.
3. These cross-protocol safeguards are generated automatically from a single high-level “redistribute” intent.

6.9 Micro-Segmentation and SGT Policies

Beyond zone-based firewalling, the policy compiler supports fine-grained micro-segmentation via Security Group Tags (SGTs) or equivalent group-based access control:

The SGT policy compiler:

1. Assigns SGT tags to node groups based on role, VLAN, or subnet membership.
2. Reads the access matrix to determine which source-destination SGT pairs are permitted or denied.
3. Generates `sgt_tag`, `sgacl_*`, and `cts_enforcement` attributes on each relevant device.
4. For devices that do not support hardware SGT enforcement, falls back to equivalent VACL or port-ACL rules with provenance annotations noting the fallback.

6.10 Compilation Complexity

Theorem 6 (Policy Compilation Complexity). Given p policies and a plan graph with n entities (nodes + endpoints), the policy compilation stage runs in $O(p \cdot n)$ time in the worst case (every policy matches every entity). In practice, match predicates use indexed lookups (e.g., hash maps on `role`), reducing the average case to $O(p + n)$.

7 Source-of-Truth Integration

Source-of-truth (SoT) systems are the authoritative record of what exists in the network: which devices are deployed, how they are connected, what IP addresses are assigned, and which VLANs are provisioned. Integrating these systems into the composition pipeline requires careful schema mapping and synchronisation.

7.1 Schema Mapping

The core challenge of SoT integration is mapping between the source-of-truth data model and the plan topology attribute schema. Figure 32 illustrates this for NetBox.

The schema mapping is defined declaratively, enabling the same adapter framework to support multiple SoT platforms:

Schema Mapping Definition

```
adapter: netbox
version: "3.7"
mappings:
  node:
    source: dcim.Device
    attributes:
      hostname: {field: name}
      role: {field: role.slug, transform: enum}
      platform: {field: platform.slug, transform: enum}
      site: {field: site.slug}
      rack: {field: rack.name}
      serial_number: {field: serial}
  endpoint:
    source: dcim.Interface
    attributes:
      interface_name: {field: name, transform: canonicalise}
      mtu: {field: mtu, default: 1500}
      admin_status: {field: enabled, transform: bool_to_admin}
  connectivity:
    source: dcim.Cable
    a_endpoint: {field: a_terminations[0].object}
    b_endpoint: {field: b_terminations[0].object}
  addressing:
    source: ipam.IPAddress
    attributes:
      ipv4_address: {field: address, filter: "family=4"}
      ipv6_address: {field: address, filter: "family=6"}
```

7.2 Incremental Synchronisation

In operational environments, the SoT changes continuously: devices are added, interfaces are re-addressed, VLANs are provisioned. The composition pipeline must support *incremental synchronisation* to avoid re-importing the entire inventory on every change.

The incremental strategy operates as follows:

1. **Initial import:** a full import at time t_0 creates the baseline SoT fragment.
2. **Change detection:** the adapter subscribes to SoT change events (webhooks, changelog API, or polling) and produces *delta fragments* Δ_i , each containing only the changed entities and attributes.
3. **Delta application:** each delta is overlaid onto the current plan using the overlay operator: $G'_{\text{plan}} = G_{\text{plan}} \triangleright \Delta_i$.
4. **Re-validation:** after applying deltas, the validation stage runs to ensure the updated plan remains consistent.

Design Rule 13: SoT as Authority

For attributes within the SoT's domain (inventory, addressing, VLAN assignment), the SoT is the highest-priority source. Policy fragments may *reference* SoT-provided attributes (e.g., matching on role) but should not *override* them.

7.3 Drift Detection

When the plan topology is used to generate device configurations, a feedback loop can detect *drift* between the intended state (G_{plan}) and the observed state (device operational data or SoT records).

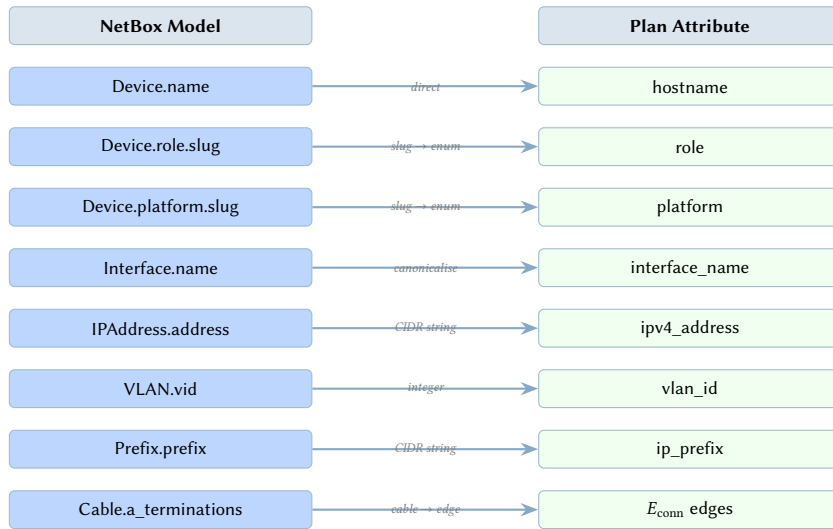


Figure 32. Schema mapping from NetBox data models to plan topology attributes. Each NetBox field maps to a plan attribute with an optional transformation (type coercion, canonicalisation, or structural mapping).

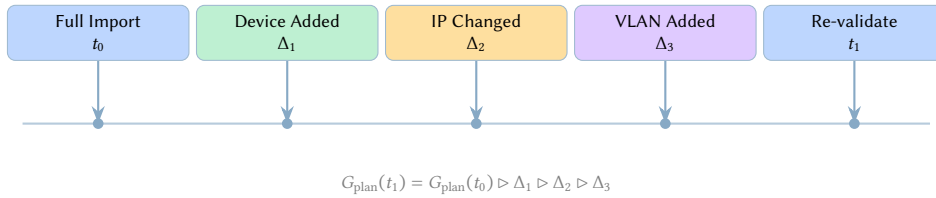


Figure 33. Incremental synchronisation timeline. After the initial full import at t_0 , subsequent changes are captured as delta fragments and overlaid onto the existing plan topology.

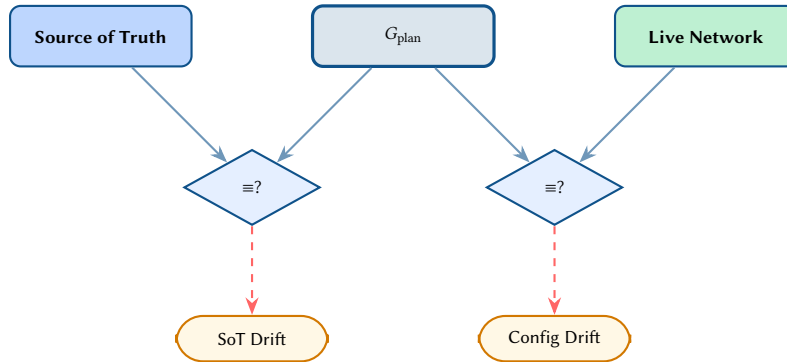


Figure 34. Drift detection. The plan topology is compared against both the source of truth and the live network to detect discrepancies in either direction.

Drift detection compares attributes in three domains (fig. 34):

- **SoT drift:** attributes in G_{plan} that differ from the SoT (e.g., a manually assigned IP that doesn't match IPAM).
- **Config drift:** attributes in G_{plan} that differ from the live device configuration (e.g., an OSPF cost that was manually changed on the device).
- **Bidirectional drift:** discrepancies between the SoT and the live network that bypass the plan entirely.

8 Operator Workflows and Tractability

While the mathematical foundations of the join-semilattice and property graph ensure rigorous determinism, a composition framework is only useful if it is tractable for network operators. This section bridges the gap between pure mathematics and daily network engineering by detailing the Domain-Specific Language (DSL) requirements and the end-to-end integration with a Network Source of Truth (NSoT).

8.1 The Role of the DSL

Operators should not interact directly with raw graph vertices or algebraic retraction operators. Instead, they require a high-level, declarative Domain-Specific Language (DSL) that abstracts these concepts into familiar network engineering terms.

To be tractable, a DSL must provide:

- **Selector Abstractions:** Instead of iterating over nodes ($v \in V$), the DSL must allow tag-based grouping (e.g., `match: role == "spine"`).
- **Thunk Primitives:** Built-in macros for dynamic allocation (e.g., `allocate_subnet(pool="campus")`) that the pipeline automatically lowers into resource promises.
- **Safety Rails:** Syntactic validation before the pipeline even attempts graph construction.

8.2 NSoT and DSL Integration Flow

The true power of the narrow-waist pipeline is realized when the DSL (expressing *Policy Intent*) is merged with an NSoT (expressing *Physical Reality and Inventory*).

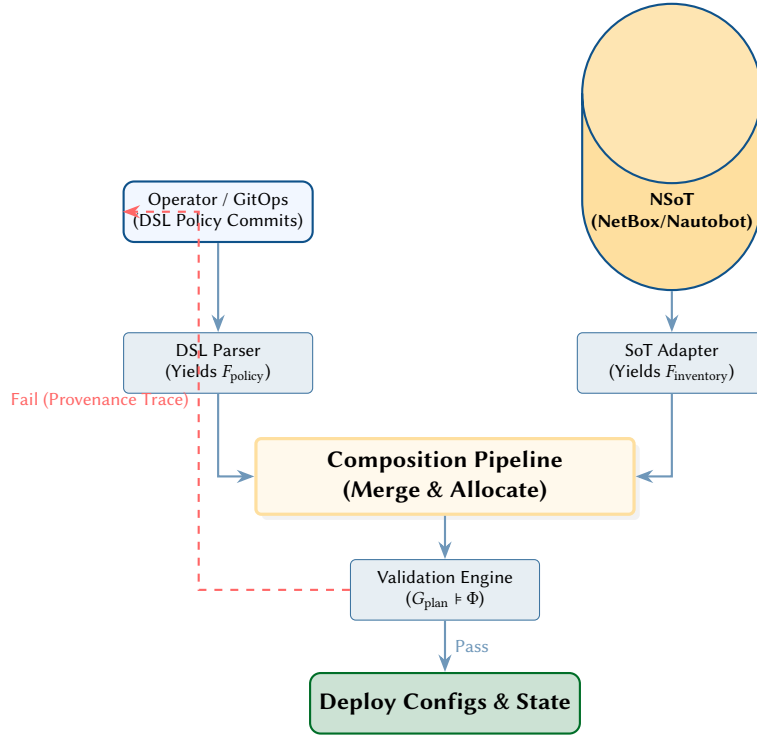


Figure 35. The Operator Workflow Flowchart. The DSL policies and NSoT inventory operate as independent lifecycles, converging only inside the Composition Pipeline. Errors route back to the operator with clear provenance.

As shown in fig. 35, the integration relies on a dual-lifecycle model:

1. **The Inventory Lifecycle (NSoT):** Rack space, cabling, and hardware RMAs are managed directly in the NSoT by datacenter technicians. The NSoT remains the ultimate source of truth for the physical graph (V and E_{conn}).
2. **The Intent Lifecycle (DSL):** Network engineers author policies (e.g., “all edge ports require 802.1X”) via the DSL in a Git repository.
3. **The Structural Lifecycle (Topological Specification):** Architects draw the high-level structural intent in a visual tool (e.g., Visio, Excalidraw, or a custom Web UI). This visual topology is exported as a serialized JSON/YAML artifact and committed to Git, acting as the structural baseline fragment (F_{struct}).
4. **Convergence:** A CI/CD pipeline triggers the Composition Engine. The NSoT Adapter extracts the current physical reality, while the DSL parser and Whiteboard parser extract the intended policies and structures. The pipeline mathematically merges them ($\oplus$), allocates necessary resources (IPs, VLANs), and validates the combined graph.

This architecture ensures tractability. Engineers do not need to manually configure thousands of switchports; they simply write a 10-line DSL policy and allow the pipeline to dynamically map it against the thousands of nodes pulled from the NSoT.

9 Worked Examples

This section presents three end-to-end worked examples that demonstrate the composition pipeline on realistic network design scenarios.

9.1 Example 1: Greenfield Data Centre Fabric

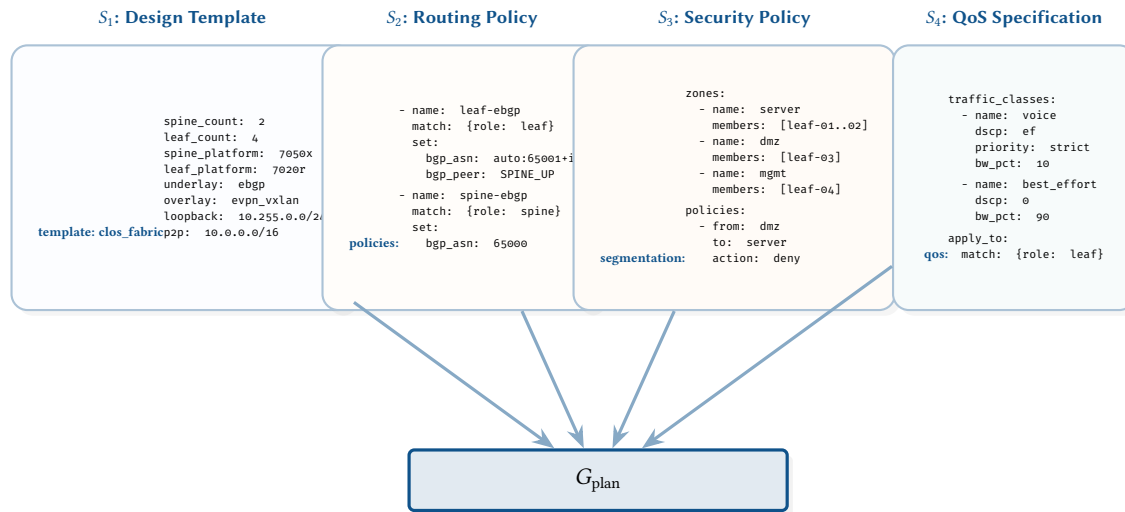


Figure 36. Greenfield DC fabric: four intent sources feed the composition pipeline to produce a complete plan topology.

9.1.1 Scenario

A new data centre requires a 2-spine, 4-leaf Clos fabric with eBGP underlay, EVPN-VXLAN overlay, and standard security and QoS policies.

9.1.2 Input Sources

Four intent sources contribute to the plan:

9.1.3 Composition Pipeline Trace

- Parse:** the template expander generates a full topology fragment F_1 with 2 spines, 4 leaves, 8 P2P links, loopback interfaces, and base attributes (platform, role, interface names). The policy, security, and QoS parsers each produce their fragments F_2, F_3, F_4 .
- Normalise:** interface names are canonicalised; the auto-increment expression in F_2 is expanded (leaf-01 → ASN 65001, ..., leaf-04 → ASN 65004).
- Merge:** fragments are composed following the DAG:

$$G' = F_1 \triangleright F_2 \triangleright F_3 \triangleright F_4$$

No conflicts arise because each source targets a different attribute namespace (topology: structural; policy: bgp_*; security: acl_*, security_zone; QoS: qos_*).

- Allocate:** the auto-increment ASN thunks are resolved (leaf-01 → 65001, ..., leaf-04 → 65004). Point-to-point subnets are allocated from 10.0.0.0/16 (eight /30 subnets for 8 links). Loopback addresses are drawn from 10.255.0.0/24. EVPN VNI identifiers are assigned from the global VNI pool.
- Default:** protocol defaults are applied—OSPF timers, BGP hold time, STP priority, MTU—to fill gaps not covered by any intent source.
- Validate:** structural checks pass (all links bidirectional, no orphans, unique ownership). Protocol checks pass (BGP peer reciprocity confirmed across all spine-leaf pairs, EVPN route targets consistent, allocated subnets non-overlapping).

9.1.4 Resulting Plan Topology

9.1.5 Case Study Analysis

This greenfield example demonstrates the fundamental power of the **Structural Composition** methodology combined with the narrow-waist architecture:

- Separation of Concerns:** The physical structural intent (the template) is completely decoupled from the routing and security intent. The security team can update their zone policies (S_3) without ever needing to touch or understand the underlying BGP underlay (S_2).
- High-Signal Intent:** The inputs are incredibly dense and human-readable. A few dozen lines of YAML generate a mathematically rigorous graph containing hundreds of exact attribute assignments.
- Pedagogical Clarity (Show, Don't Tell):** Rather than forcing the operator to read raw generated device configurations to understand the network's purpose, the intent sources themselves serve as the authoritative documentation of *why* the network exists. This pedagogical progression dramatically reduces the cognitive load for junior engineers trying to understand the fabric.

9.2 Example 2: Brownfield Migration

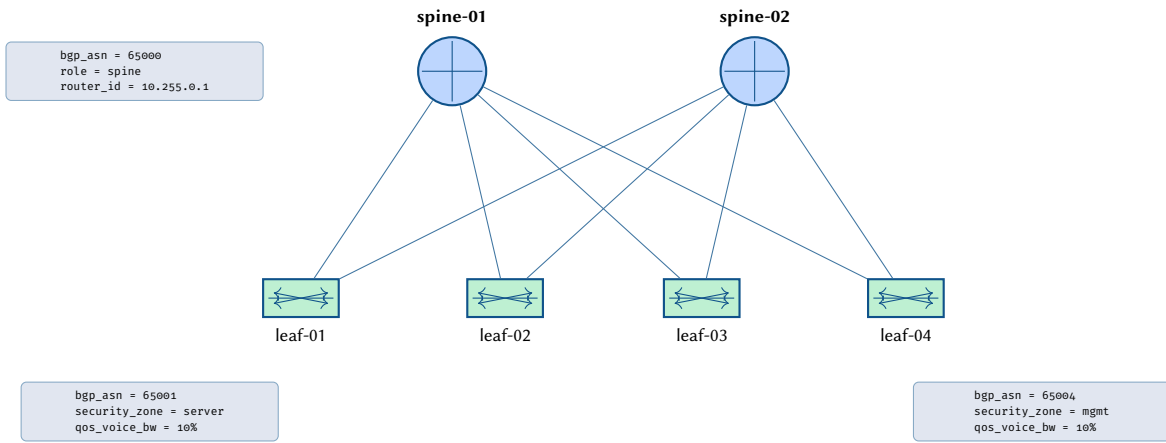


Figure 37. Resulting plan topology for the greenfield DC fabric. Each node carries attributes contributed by multiple intent sources: structural attributes from the template, routing from the policy, segmentation from security, and traffic classification from QoS.

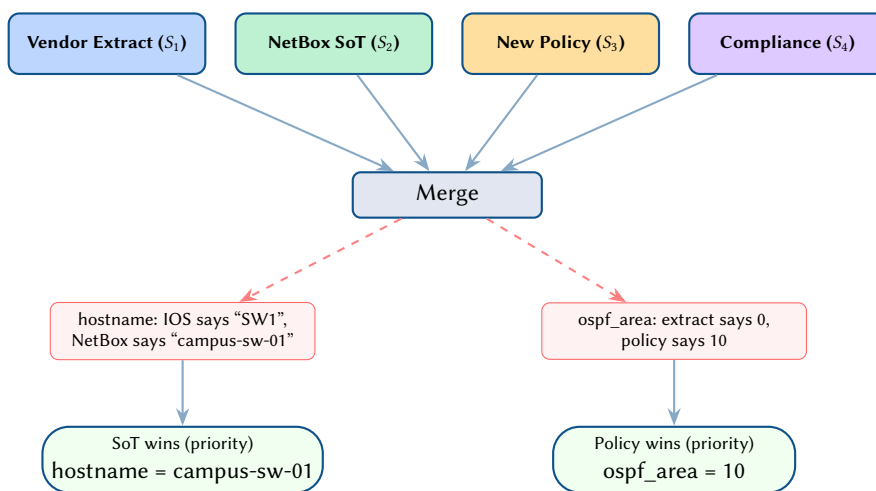


Figure 38. Brownfield migration with conflict resolution. The vendor extract and NetBox disagree on hostnames (SoT wins); the vendor extract and new policy disagree on OSPF area (policy wins, as it represents the migration intent).

9.2.1 Scenario

An existing campus network running on Cisco IOS is being migrated to a new automation platform. The existing configurations must be captured, normalised, and enhanced with new policies.

9.2.2 Input Sources

1. **Vendor config extraction** (S_1): the existing IOS configurations from 12 switches and 4 routers are parsed and normalised into a plan fragment containing all observed attributes.
2. **NetBox SoT** (S_2): the device inventory and IPAM records are imported, providing the authoritative hostnames, roles, and IP assignments.
3. **New routing policy** (S_3): the migration plan includes switching from OSPF single-area to multi-area OSPF, so a policy fragment reassigns `ospf_area` attributes.
4. **Compliance** (S_4): a compliance check enforces SNMPv3-only, SSH-only, and encrypted syslog.

9.2.3 Composition with Conflict Resolution

This example demonstrates the conflict resolution mechanism:

- **Hostname conflict:** the IOS config uses short names (“SW1”), NetBox uses canonical names (“campus-sw-01”). The SoT has higher priority for inventory attributes, so “campus-sw-01” wins.
- **OSPF area conflict:** the existing config has all interfaces in area 0; the migration policy reassigns distribution switches to area 10. The policy source has higher priority for routing attributes (it represents the desired future state), so area 10 wins.
- **No conflict on compliance:** the compliance source only sets attributes (`snmpv3_enabled`, `ssh_enabled`) that no other source touches.

9.2.4 Full Pipeline Trace

1. **Parse:** vendor configs are parsed via the IOS grammar; NetBox records are fetched via the SoT adapter; the routing policy and compliance files are read by the YAML parser.

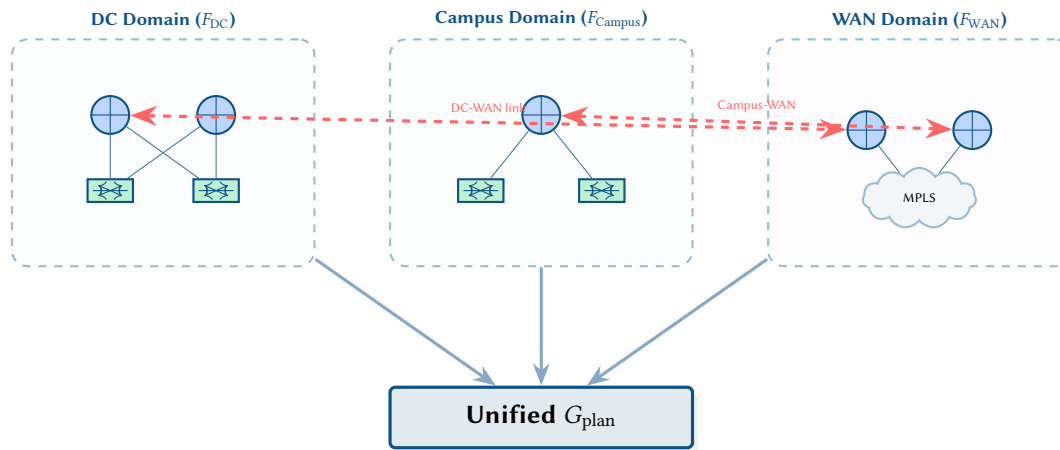


Figure 39. Multi-domain composition. Three independently managed network domains are composed into a unified plan topology. Cross-domain links (dashed red) are established via matching endpoint identifiers at domain boundaries.

2. **Normalise:** IOS interface names (e.g., GigabitEthernet0/1) are canonicalised; SNMP community strings from the vendor extract are mapped to `snmpv3_*` attributes.
3. **Merge:** the four fragments are composed. Hostname and OSPF area conflicts are resolved by priority as described above.
4. **Allocate:** the new multi-area OSPF design requires updated loopback addresses for area border routers. The allocator assigns loopbacks from a dedicated ABR pool (10.200.0.0/24).
5. **Default:** platform defaults (IOS-specific timers, MTU) are applied; SNMPv3 engine IDs are derived from device serial numbers.
6. **Validate:** OSPF area boundary checks confirm that ABRs have interfaces in both areas; compliance checks confirm SSH-only and SNMPv3-only across all 16 devices.

9.2.5 Case Study Analysis

Brownfield environments represent the true test of any network modeling framework. The power of this approach lies in its mathematical safety:

- **The Anchor of Reality:** By pulling existing vendor configurations (S_1) into the same flat-attribute G_{plan} schema, operators have an objective mathematical baseline.
- **Gradual Intent Introduction (Mitigating "Big Bang" Risk):** Operators do not need to rewrite their entire network in a high-level DSL overnight. They can selectively overlay new intent (S_3) onto the legacy baseline, allowing the merge priority system to deterministically override the old state. This effectively eliminates the risk of catastrophic "Big Bang" migrations.
- **Verification Before Action:** The compliance and validation gates run on the *composed graph*, proving that the proposed migration state (the new multi-area OSPF design) is sound before a single line of CLI is ever pushed to the physical devices.

9.3 Example 3: Multi-Domain Composition

9.3.1 Scenario

A large enterprise has three network domains—data centre, campus, and WAN—each managed by a different team with different tools. The goal is to compose a unified plan topology spanning all three domains.

9.3.2 Domain Boundary Handling

The key challenge in multi-domain composition is establishing connectivity across domain boundaries. Each domain team defines their internal topology independently, but the boundary interfaces must match:

Cross-Domain Link Specification

```
# Defined by the WAN team
cross_domain_links:
  - local: pe-01:Ethernet1
    remote: dc-spine-01:Ethernet48
    description: "DC-to-WAN uplink"

  - local: pe-02:Ethernet1
    remote: campus-core:Ethernet49
    description: "Campus-to-WAN uplink"
```

During the merge stage, these cross-domain link specifications create E_{conn} edges between endpoints that belong to different domain fragments. The composition pipeline verifies that both endpoints exist and that their physical attributes (speed, MTU) are compatible.

9.3.3 Per-Domain Policy Autonomy

Each domain retains its own policy sources:

- The **DC team** defines eBGP underlay, EVPN overlay, and DC QoS policies.
- The **Campus team** defines OSPF areas, VLAN assignments, and 802.1X policies.
- The **WAN team** defines MPLS VPN, traffic engineering, and inter-site QoS.

These policies are composed independently within each domain and then merged into the unified plan. Cross-domain policies (e.g., end-to-end QoS DSCP preservation) are defined as separate intent sources that reference entities from multiple domains.

9.3.4 Multi-Domain Pipeline Trace

1. **Parse:** each domain team produces its own fragment (F_{DC} , F_{Campus} , F_{WAN}) using their preferred tools and formats. Cross-domain link specifications are parsed as a fourth source.
2. **Normalise:** interface names and attribute keys are harmonised across the three vendor ecosystems (Arista EOS in the DC, Cisco IOS on campus, Juniper JunOS on the WAN).
3. **Merge:** domain fragments are unioned; cross-domain link specifications create E_{conn} edges at the boundaries. Cross-domain QoS policy overlays DSCP-preservation attributes onto boundary interfaces.
4. **Allocate:** the WAN team's MPLS VPN requires VRF route distinguishers and route targets. The allocator assigns RD values from a global pool, ensuring uniqueness across all three domains. Inter-domain BGP peering addresses are allocated from a shared /24 pool.
5. **Default:** per-domain platform defaults are applied (EOS defaults for DC, IOS defaults for campus, JunOS defaults for WAN). Cross-domain links inherit the most conservative MTU from either side.
6. **Validate:** cross-domain boundary checks verify MTU agreement, BGP peer reciprocity across domain boundaries, and consistent DSCP markings on both sides of every cross-domain link.

9.3.5 Case Study Analysis

Multi-domain networks frequently suffer from organizational silos, where a failure at the boundary requires days of cross-team coordination.

- **Federated Autonomy:** The composition pipeline allows each domain to maintain absolute autonomy over their own intent sources and tooling. The DC team uses templates; the Campus team uses manual topologies.
- **The Narrow Waist Guarantee:** Because all domain fragments compile down to the exact same G_{plan} schema, the merged graph seamlessly connects disparate vendor ecosystems (EOS, IOS, JunOS) and disparate operational models.
- **Cross-Domain Invariants (Contractual Boundaries):** Structural assertions and design rules (e.g., MTU matching) are enforced precisely at the "glue" points between the domains. These boundaries act as strictly enforced mathematical contracts between organizational teams, catching the exact misconfigurations that traditionally cause catastrophic multi-domain outages.

9.4 Example 4: Day 2 Operations - Security Remediation

9.4.1 Scenario

A zero-day vulnerability is announced for a specific class of database servers. The security team needs to instantly quarantine these servers across the entire data centre fabric without disrupting other tenant traffic.

9.4.2 Composition Pipeline Trace

1. **Intent Update:** The security team does not log into 50 individual switches to modify ACLs. Instead, they update a single Security Primitive in the policy DSL (S_3): they add a Deny assertion from all zones to the DB_Servers zone.
2. **Re-Composition:** The CI/CD pipeline triggers a re-compile. The new security fragment (F'_3) is merged with the existing topology (F_1), routing (F_2), and QoS (F_4) fragments.
3. **Graph Calculation:** The composition engine automatically calculates the "blast radius" by tracing the graph edges to find exactly which leaf switches host the compromised servers.
4. **Validation:** The pipeline verifies that the new quarantine policy does not conflict with infrastructure assertions (e.g., management and control-plane traffic are still permitted).

9.4.3 Case Study Analysis

This Day 2 operational scenario highlights the agility of the Structural Composition model:

- **Surgical Precision:** By relying on dynamic match predicates and topological assertions, the pipeline calculates exactly which endpoints require updated state. Operators avoid the risk of fat-fingering a global ACL.
- **Speed to Remediation:** A single, declarative intent update is algebraically expanded into hundreds of distributed configuration lines within seconds, safely executing a fabric-wide quarantine.

9.5 Example 5: Hardware Refresh / Vendor Swap

9.5.1 Scenario

Due to global supply chain issues, the organization decides to swap their aging Cisco campus edge switches for Arista or Juniper hardware.

9.5.2 Composition Pipeline Trace

1. **Inventory Update:** The physical technician updates the NSoT (S_2) to reflect the new switch models and their interface layouts.
2. **Zero Intent Change:** The network architect *does not* change the policy DSL (S_3) or The Structural Baseline structural topology (S_1). The high-level intent ("this edge port requires 802.1X and belongs to VLAN 10") remains identical.
3. **Re-Composition:** The pipeline merges the existing intent with the updated NSoT inventory. The resulting G_{plan} is structurally identical to the previous version, except the platform attributes on the edge nodes now reflect the new vendor.
4. **Downstream Rendering:** The functorial overlay mappings ($\mathcal{F}$) process the exact same G_{plan} . However, the final translation layer reads the new platform attribute and seamlessly generates EOS or JunOS configurations instead of IOS.

9.5.3 Case Study Analysis

This is the ultimate realization of the Intent-Based Networking promise:

- **Vendor Agnosticism:** The "Narrow Waist" architecture proves its value. Because the Intent Layer (S_i) is completely decoupled from the Realisation Layer (the downstream renderers) via G_{plan} , the operator's intellectual property—their architectural design—is protected from vendor lock-in.
- **Preservation of Mathematical Intent:** The algebraic composition ensures that the logical state of the network survives the physical hardware swap untouched.

10 Validation and Verification

Validation is the final gate of the composition pipeline, ensuring that the composed plan topology is internally consistent, complete, and ready for functor consumption. This section extends the validation framework of ANK-TR-2026-04 with composition-specific checks.

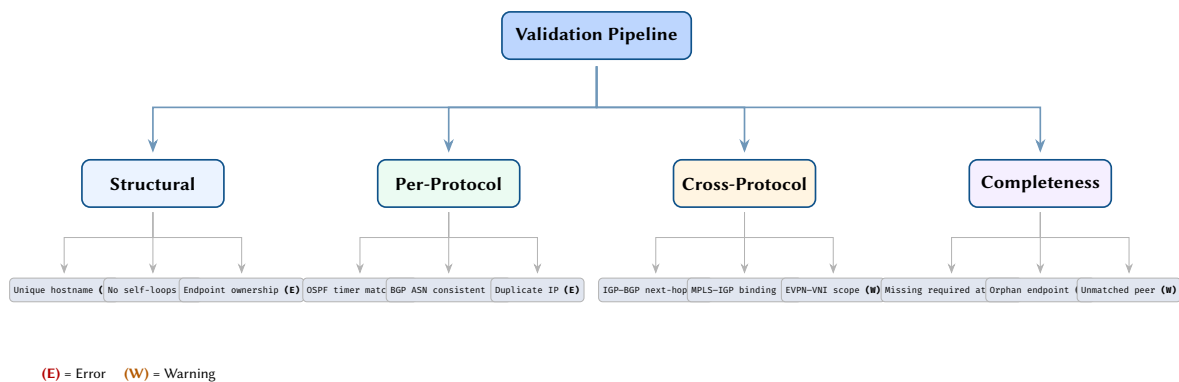


Figure 40. Taxonomy of validation rules. Each rule is tagged with its severity level: (E) errors block functor execution, (W) warnings are reported but execution proceeds. Representative rules are shown; the full set is defined in §10.

10.1 Composition-Specific Validation Rules

Beyond the structural, protocol, and completeness checks defined in ANK-TR-2026-04 (and summarized in fig. 40), the composition pipeline introduces additional validation rules specific to the multi-source composition context.

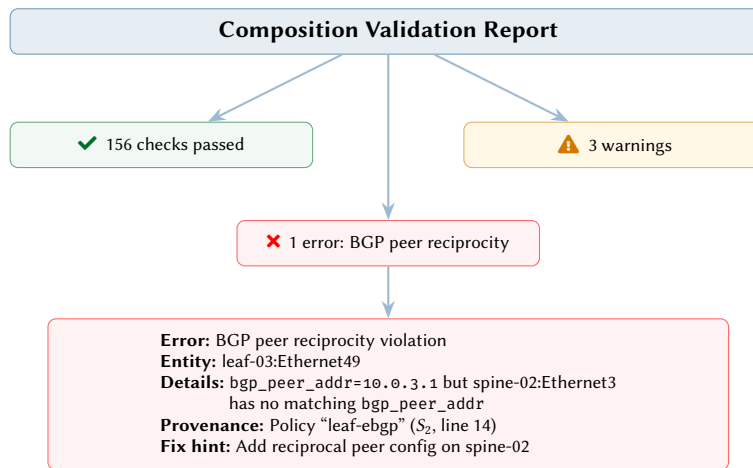


Figure 41. Structure of a composition validation report. Errors include the failing rule, affected entities, provenance trace to the responsible intent source, and a fix hint.

10.1.1 Provenance Consistency

Design Rule 14: No Orphan Attributes

Every attribute in the composed plan must be traceable to at least one intent source. Attributes that appear in the plan without a provenance record indicate a bug in the composition pipeline.

10.1.2 Cross-Fragment Integrity

Design Rule 15: Boundary Consistency

For every cross-fragment connectivity edge (e_1, e_2) where e_1 and e_2 originate from different fragments, the following must hold:

- Both endpoints agree on link-level attributes (speed, MTU, duplex).
- If both endpoints have IP addresses, they must be in the same subnet.
- If both endpoints enable a routing protocol, their protocol parameters (area, ASN, timers) must be compatible.

10.1.3 Policy Completeness

Design Rule 16: No Unresolved Generators

After composition, no attribute value may contain an unresolved generator expression (e.g., `auto:65001+index` must have been expanded to a concrete integer). Unresolved generators indicate that the policy compiler failed to evaluate the expression.

10.2 Validation Report Structure

The validation report (fig. 41) includes:

1. **Summary:** counts of passed checks, warnings, and errors.
2. **Per-error detail:** the failing rule, affected entities, the specific attribute values that caused the failure, provenance traces, and suggested fixes.
3. **Per-warning detail:** non-fatal issues such as unused attributes, shadowed ACL rules, or suboptimal configurations.
4. **Coverage report:** which intent sources contributed to which parts of the plan, highlighting any “unclaimed” entities that were produced by a single source without cross-validation.

10.3 Provenance Trace Visualisation

The provenance trace (fig. 42) records, for every attribute in the composed plan:

- The **intent source** that asserted the value (including file path and line number for file-based sources, or API endpoint and record ID for SoT sources).
- The **pipeline stage** at which the value was written (merge, allocate, or default).
- Whether the value **won a conflict** and, if so, which other source was overridden and why.

This information enables “git-blame-style” auditing of the plan topology: for any attribute, an operator can immediately determine who set it, when, and why.

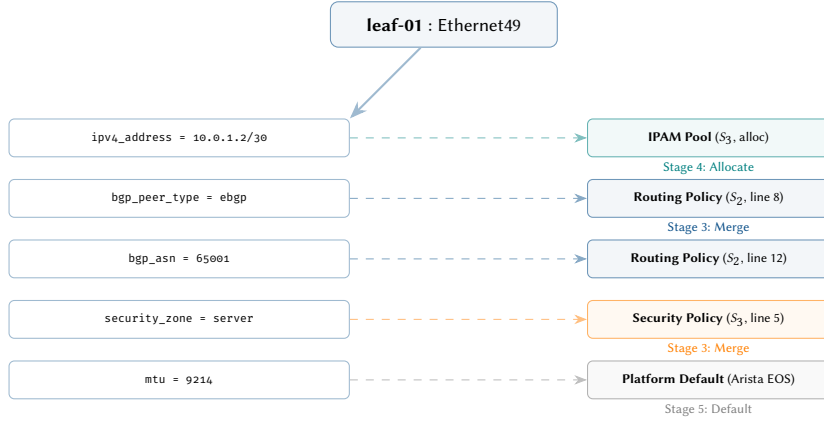


Figure 42. Provenance trace for a single entity. Every attribute on leaf-01:Ethernet49 is traced back to the intent source, source line, and pipeline stage that produced it.

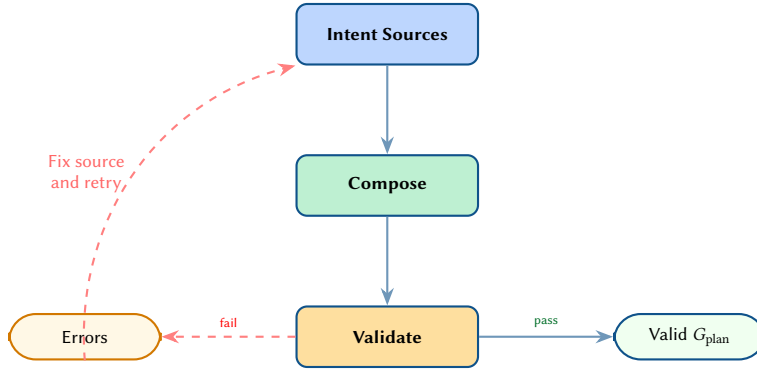


Figure 43. Iterative composition. Validation errors trigger a feedback loop: the operator fixes the responsible intent source and re-runs the pipeline.

10.4 Iterative Composition

In practice, the first composition attempt rarely produces a valid plan on the first try. The composition pipeline supports an iterative workflow (fig. 43):

1. Run the composition pipeline.
2. Review validation errors with their provenance traces.
3. Fix the responsible intent source(s).
4. Re-run the pipeline (incrementally, reusing cached fragments from unchanged sources).
5. Repeat until validation passes.

This iterative cycle is analogous to the edit–compile–fix cycle in software development. The provenance traces in validation errors serve the same role as compiler error messages with source locations: they tell the operator exactly which intent source to fix.

11 Implementation Considerations

This section discusses practical considerations for implementing the composition pipeline in the Network Topology Engine (NTE) framework.

11.1 Architecture Overview

The implementation (fig. 44) follows a plugin architecture:

- **Parser plugins:** each input modality is implemented as a plugin conforming to a common interface ($\text{parse}(S_i) \rightarrow F_i$). New modalities can be added by implementing this interface.
- **Core engine:** the normaliser, merger, and defaulter are implemented as a fixed pipeline within the core engine. The merger uses the fragment algebra operators defined in section 5.
- **Provenance store:** a database (or in-memory store) recording, for each attribute in the composed plan, which source produced it and at what priority.
- **Validator:** implements the three-phase validation from ANK-TR-2026-04 plus the composition-specific rules from section 10.

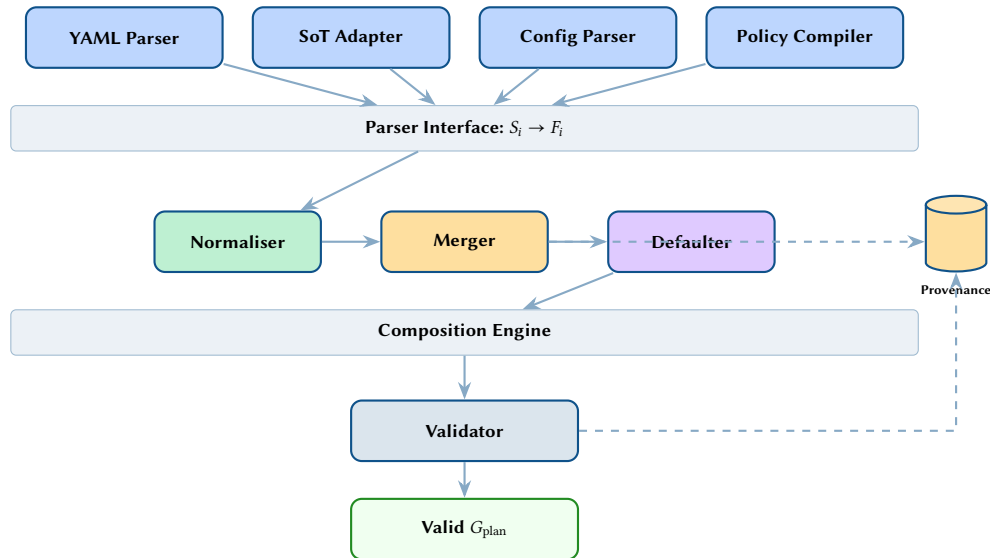


Figure 44. Implementation architecture. Parsers implement a common interface producing fragments. The core engine handles normalisation, merging, and defaulting. A provenance store tracks which source produced each attribute.

11.2 Performance Considerations

Design Decision 1: Parallelism in the Pipeline

The parse stage is embarrassingly parallel (each source parsed independently). The normalise stage is per-fragment parallel. The merge stage is sequential across DAG levels but parallel within each level. The default and validate stages operate on the single merged graph and are sequential, but individual checks within validation can run in parallel.

For large networks (thousands of devices), the dominant cost is in the merge stage, where attribute maps must be combined across many fragments. Using hash maps for attribute storage and indexed lookups for policy matching keeps the merge stage within $O(n \cdot k)$ where n is the number of entities and k is the average number of contributing fragments per entity.

11.3 Extensibility

The composition framework is designed for extensibility along three axes:

1. **New input modalities:** adding a new parser plugin requires implementing only the parse interface. No changes to the core engine, merger, or validator are needed.
2. **New attribute namespaces:** when new protocol families are added to the plan topology schema (per ANK-TR-2026-04), the composition pipeline handles them automatically—the merger and defaulter operate on arbitrary key-value pairs.
3. **New validation rules:** composition-specific validation rules can be registered as plugins, enabling domain-specific checks (e.g., an EVPN-specific validator that checks route target consistency across all VTEPs).

11.4 Configuration Management

The composition pipeline itself is configured via a *composition manifest* that declares the intent sources, their priorities, and the conflict resolution strategy:

Composition Manifest

```

composition:
  name: "dc1-fabric"
  version: "2.1"

sources:
  - name: topology
    type: template
    path: templates/clos-2x4.yaml
    priority: 10

  - name: ipam

```

```

type: netbox
url: https://netbox.example.com
token_env: NETBOX_TOKEN
priority: 20

- name: routing
  type: policy
  path: policies/dc1-routing.yaml
  priority: 50

- name: security
  type: policy
  path: policies/dc1-security.yaml
  priority: 80

- name: compliance
  type: compliance
  path: policies/enterprise-compliance.yaml
  priority: 100

conflict_resolution: priority
default_strategy: protocol-then-platform

```

The manifest serves as the single entry point for the composition pipeline and is version-controlled alongside the intent sources, enabling reproducible plan generation.

12 Critical Assessment: Risks and Challenges

While the narrow-waist architecture provides a rigorous framework for network intent composition, its practical application faces several non-trivial challenges. To maintain architectural integrity, these risks must be addressed with transparency and critical discipline.

12.1 The Complexity of Fragment Algebra

The theoretical properties of the fragment algebra—associativity and commutativity (section 5)—are essential for declarative GitOps workflows. However, maintaining these properties in the presence of *stateful, order-dependent overrides* is difficult. For example, security policies (ACLs) and routing policy chains (route maps) are inherently ordered. If two independent fragments attempt to insert rules into the same ordered chain, the result of a simple union may be non-deterministic or semantically incorrect.

Mitigation: The composition pipeline must enforce a strict *decomposition* of ordered structures into independent, tagged primitives. Rather than merging raw ACL strings, the pipeline merges independent rule objects that are re-ordered by a deterministic post-merge sorting function based on architectural specificity.

12.2 The “Everything is an Attribute” Fallacy

The decision to represent all protocol state as flat attributes on a property hypergraph (ANK-TR-2026-03) simplifies the merger but risks semantic dilution. Flattening complex, hierarchical data models (like YANG trees or nested JSON objects) into a flat namespace (`ospf_*`, `bgp_*`) can lead to a “bloated waist.” If the schema grows to thousands of attributes, the cognitive load on intent-source authors increases, and the risk of naming collisions or attribute misuse rises.

Mitigation: The Schema Reference (ANK-TR-2026-04) must be strictly partitioned into namespaces, and the Policy Compiler (section 6) must serve as the primary interface, shielding operators from the raw attribute schema.

12.3 Conflict Resolution and Priority Creep

In a multi-stakeholder environment, priority-based conflict resolution (section 2.3) can lead to “Priority Creep.” If different teams (Security, Operations, Compliance) all mark their intent sources as `priority: 100` to ensure their requirements are met, the pipeline reverts to non-deterministic behaviour or rejection.

Mitigation: Priority assignments must be governed by an *architectural policy* rather than individual sources. The Composition Manifest (section 11) should define immutable priority tiers based on the *role* of the intent source, ensuring that Compliance always overrides Operations, and explicitly-set manual overrides take precedence over automated templates.

12.4 The Verification Gap

A valid plan topology, even one that passes all three phases of validation (section 10), does not guarantee a functioning physical network. The **verification gap** exists between the mathematical end-state and the

physical realisation. Bugs in the Network Config Compiler, undocumented vendor hardware limitations (e.g., TCAM exhaustion, ASIC buffer constraints), or environmental failures can still result in an outage.

Mitigation: Composition must not be viewed as a one-way street. The architecture must incorporate a **drift-detection loop** (fig. 34) that compares the intended G_{plan} against the observed operational state extracted from the network.

12.5 Provenance and Metadata Overhead

Maintaining a full audit trail for every attribute (ANK-TR-2026-06) adds significant metadata overhead. In a network with 10^5 endpoints and 10^6 attributes, the provenance store can easily exceed the size of the actual plan topology. Querying this metadata during large-scale re-compositions can impact pipeline latency.

Mitigation: Provenance should be handled as an *offline* metadata store, indexed by attribute ID but not necessarily resident in the hot path of the composition merger. The pipeline should use compact bitmasks or references to source IDs to keep the in-memory graph representation lean.

13 Related Work

The composition pipeline addresses the challenge of bridging high-level operator intent with low-level network state. This problem sits at the intersection of Intent-Based Networking (IBN), network verification, formal languages, and standardized data modeling. In this section, we survey the relevant literature from both academia (e.g., SIGCOMM, NSDI) and industry (IETF, OpenConfig) to position our narrow-waist architecture.

13.1 Intent-Based Networking and Configuration Synthesis

The goal of Intent-Based Networking is to allow operators to declare *what* they want the network to do, relying on a compiler or orchestrator to figure out *how* to configure the underlying devices.

Early work by Lumi et al. [2] proposed a tiered architecture (intent, translation, activation) that structurally mirrors our pipeline. However, Lumi focuses strictly on natural language translation for a single, homogeneous intent layer. Porter et al. [3] identify the “intent translation” stage as the critical bottleneck in multi-layer networks, which is precisely the problem our composition algebra formalizes.

Within the academic networking community, several domain-specific synthesis languages have been proposed:

- **Propane** [4]: Propane provides a high-level routing language that compiles down to per-device BGP route-map configurations. While highly effective, Propane addresses a single protocol (BGP) with extreme depth. Our framework, conversely, provides a generalized mathematical model for *all* protocols across the stack.
- **Merlin** [5]: Merlin is a declarative language for expressing bandwidth and path constraints, compiling them into OpenFlow forwarding rules. Merlin operates at the data-plane and forwarding-plane level. Our pipeline abstracts the *configuration and protocol plane*, leaving forwarding-state resolution to downstream protocol execution.

Commercial IBN systems (such as Cisco DNA Center or Juniper Apstra) tackle the composition problem by forcing users into proprietary intent models. Our approach differs by defining G_{plan} as an open, vendor-neutral Intermediate Representation (IR). By defining a mathematically rigorous merge operator ($\oplus$), we allow diverse input modalities to interoperate without vendor lock-in.

13.2 Topological Specification and Structural Intent

Traditional Intent-Based Networking often assumes that intent is exclusively textual, relying heavily on YAML DSLs or natural language processing. However, network architecture is inherently spatial and structural; architects reason about networks through topological graphs long before writing code. Our methodology formalises this **structural intent**.

While structural network modeling has historical precedents (e.g., early graph-based simulators like OPNET or GNS3), those tools tightly coupled the representation to the simulation engine or vendor-specific VM instances. In contrast, our approach treats the structural baseline or topological fragment as just another intent modality—a pure intent source (S_i)—that is decoupled from the execution engine. This separation allows the spatial structural specification to be merged with non-spatial policy intents (like security matrices or QoS SLAs) through our algebraic pipeline, providing a level of categorical composability absent in traditional monolithic simulators.

13.3 Network Verification vs. Constructive Composition

A significant body of work has focused on formally verifying network correctness. Tools like **Batfish** [6, 7], **Minesweeper** [8], and **Header Space Analysis** [9] ingest low-level device configurations (e.g., Cisco IOS, Juniper JunOS) and build data-plane models to verify reachability or security invariants.

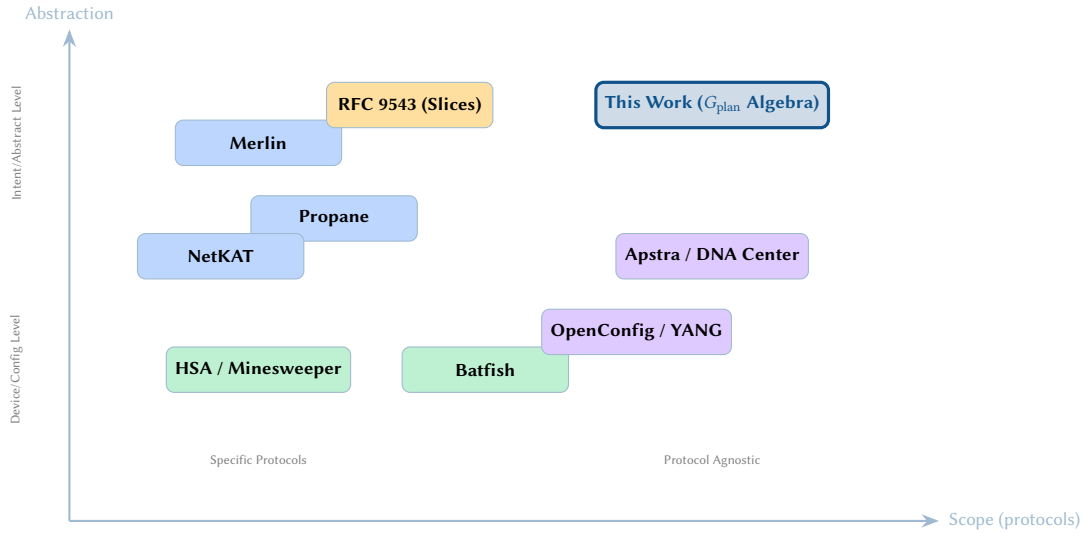


Figure 45. Related work landscape. This work occupies the high-abstraction, broad-scope quadrant. It raises the level of abstraction from post-hoc config verification (Batfish) and single-protocol compilation (Propane) to generalized algebraic composition across all network protocols.

These tools are fundamentally *post-hoc verifiers*: they analyze state *after* the composition and configuration generation processes have occurred. If an error is found, the operator must manually trace the failure back through their templates or policy DSLs.

In contrast, our Stage 6 validation engine acts as a *constructive verifier*. We enforce constraints (such as MTU matching, IP uniqueness, and structural connectivity) directly on the abstract intent graph G_{plan} *before* a single line of vendor configuration is generated. This “shift-left” approach catches architectural conflicts (e.g., a security policy conflicting with a routing policy) during the composition merge phase, preserving strict provenance so the operator knows exactly which intent source caused the violation.

Formal algebras like **NetKAT** [10] provide semantic foundations for analyzing packet trajectories. While NetKAT is ideal for software-defined networking (SDN) policy composition, our fragment algebra targets the declarative configuration state of distributed routing protocols.

13.4 Standardized Data Models (IETF and OpenConfig)

The industry has made massive strides in standardizing the *schema* of network state. The IETF’s YANG data modeling ecosystem [11] and the associated network topology models (RFC 8345 [12], RFC 8346 [13]) provide rigorous, vendor-neutral ontologies. Similarly, **OpenConfig** [14] provides widely adopted vendor-neutral YANG models for individual device configurations.

Furthermore, recent IETF efforts like the Network Slice Intent framework (RFC 9543 [15]) attempt to standardize high-level intent requests.

While these RFCs define *how data should be structured*, they do not define *how disparate intents should be mathematically combined*. A YANG model provides the alphabet, but the composition pipeline provides the grammar and algebra. Our G_{plan} property graph acts as an overlay that can easily map onto RFC 8345 topologies, but it is the join-semilattice merge operations ($\oplus$) and Thunk allocations that solve the distributed composition problem.

13.5 Category Theory and Graph Transformation

Finally, our decision to formalize plan fragments as property graphs and merge them algebraically is rooted in categorical approaches to systems engineering.

Baez and Fong [16] developed a compositional framework for open networks using “decorated cospans.” This theory formalizes how sub-networks with explicit boundaries can be glued together. Our pipeline’s cross-domain connectivity merges (where an endpoint in a Campus fragment is glued to an endpoint in a WAN fragment) is a practical, applied realization of this categorical gluing.

Furthermore, the algebraic theory of graph transformation, particularly the Double-Pushout (DPO) approach defined by Ehrig et al. [17], provides the underlying machinery for how we manipulate graphs without violating their structural integrity. By treating intent sources not as text files but as mathematical subgraphs that are pushed onto a base topology, we eliminate the silent overwrites and race conditions that plague traditional script-based automation.

14 Conclusion

14.1 Summary

This report addresses the *composition problem* for the plan topology G_{plan} : how to construct a single, consistent, vendor-neutral network intent graph from the diverse sources that network engineers actually work with.

The key contributions are:

1. A **narrow-waist architecture** that decouples upstream intent expression from downstream configuration rendering, with G_{plan} as the universal intermediate representation.
2. A taxonomy of **eight input modalities**—topology fragments, design templates, policy intent, source-of-truth databases, vendor configuration extraction, security segmentation, QoS specifications, and compliance constraints—spanning four categories (structural, policy, data, legacy).
3. A six-stage **composition pipeline** (parse, normalise, merge, allocate, default, validate) that transforms heterogeneous inputs into a valid plan topology with determinism, incremental re-composition, and provenance traceability.
4. A **fragment algebra** with three operators (union, overlay, override) and well-characterised algebraic properties (commutativity, associativity, idempotence) that govern how partial plan graphs are combined.
5. A **policy compilation** framework supporting match predicates, attribute generators, cross-device policies, and priority-based evaluation ordering.
6. A practical **operator workflow** mapping that anchors the mathematical algebra to tangible GitOps CI/CD pipelines, demonstrating how declarative DSLs and NSoT inventories seamlessly converge.
7. Three **worked examples** demonstrating greenfield design, brownfield migration, and multi-domain composition.

14.2 The Narrow Waist in Practice

The narrow-waist principle has a profound practical implication: *teams can independently evolve their intent expression tools without coordinating with the teams that consume the plan*. The DC team can adopt a new topology DSL; the security team can switch to a different policy engine; the WAN team can integrate a new SoT platform. As long as each tool produces valid plan fragments, the composition pipeline absorbs the diversity.

This mirrors the success of IP in the Internet architecture: IP’s stability at the waist enabled explosive innovation in both applications (above) and link technologies (below). We anticipate that G_{plan} ’s stability will similarly enable innovation in both intent expression and configuration rendering.

Mathematical Rigor as the Foundation of Trust. Ultimately, network operators will only stop relying on manual CLI verification if they implicitly trust the compilation engine. The algebraic composition model—underpinned by join-semilattices, structural Assertions, and Double-Pushout (DPO) transformations—provides this trust. By ensuring strict provenance tracing for every generated attribute, the pipeline guarantees that the network’s mathematical blueprint is a perfect, auditable reflection of the engineer’s original structural intent.

14.3 Relationship to the Report Series

Together with ANK-TR-2026-03 (which defines G_{plan} and its downstream consumption via functorial mappings) and ANK-TR-2026-04 (which catalogues the attribute schema and validation rules), this report completes the lifecycle of the plan topology:

Compose (this report) $\rightarrow$ **Validate** (TR-04) $\rightarrow$ **Consume** (TR-03)

14.4 Future Work

Several directions remain for future investigation:

- **Formal verification of composition:** proving that specific composition configurations (sets of sources with given priorities) always produce valid plans, eliminating the need for runtime validation in certified cases.
- **Interactive composition:** a visual tool that lets operators compose fragments interactively, seeing the merged plan update in real time as sources are added or modified.
- **Temporal composition:** extending the framework to handle time-sequenced intent (e.g., “at 02:00, switch to maintenance routing policy”) by adding temporal attributes to fragments.
- **Machine-learned defaults:** training the defaulting stage on historical network configurations to suggest context-appropriate defaults rather than fixed protocol values.
- **Natural language intent:** adding a natural language parser as an input modality, allowing operators to express intent in prose that is compiled to policy fragments.
- **Conflict explanation:** when the pipeline rejects a composition, automatically generating a human-readable explanation of why the conflict exists and suggesting minimal changes to resolve it.

References

- [1] J. Postel, *Internet protocol*, RFC 791, 1981.
- [2] A. S. Jacobs, R. J. Pfitscher, R. A. Ferreira, and L. Z. Granville, “Hey, lumi! using natural language for intent-based network management,” in *Proc. USENIX ATC*, 2021.
- [3] M. A. Porter, *What is a multilayer network?* Notices of the American Mathematical Society, 2018.
- [4] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, “Don’t mind the gap: Bridging network-wide objectives and device-level configurations,” in *Proc. ACM SIGCOMM*, 2016.
- [5] R. Soulé et al., “Merlin: A language for provisioning network resources,” in *Proc. CoNEXT*, 2014.
- [6] A. Fogel et al., “A general approach to network configuration analysis,” in *Proc. USENIX NSDI*, 2015.
- [7] A. Brown, A. Fogel, et al., “Lessons from the evolution of the batfish configuration analysis tool,” in *Proc. ACM SIGCOMM*, 2023.
- [8] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, “A general approach to network configuration verification,” in *Proc. ACM SIGCOMM*, 2017.
- [9] P. Kazemian, G. Varghese, and N. McKeown, “Header space analysis: Static checking for networks,” in *Proc. USENIX NSDI*, 2012.
- [10] C. J. Anderson et al., “NetKAT: Semantic foundations for networks,” in *Proc. POPL*, 2014.
- [11] M. Bjorklund, *The YANG 1.1 data modeling language*, RFC 7950, 2016.
- [12] A. Clemm, J. Medved, R. Varga, N. Bahadur, H. Ananthakrishnan, and X. Liu, *A YANG data model for network topologies*, RFC 8345, 2018.
- [13] A. Clemm, J. Medved, R. Varga, N. Bahadur, H. Ananthakrishnan, and X. Liu, *A YANG data model for layer 3 topologies*, RFC 8346, 2018.
- [14] OpenConfig Working Group. “OpenConfig: Vendor-neutral network configuration models.” [Online]. Available: <https://openconfig.net/>
- [15] A. Farrel et al., *A framework for IETF network slices*, RFC 9543, 2024.
- [16] J. C. Baez and B. Fong, “A compositional framework for passive linear networks,” *Theory and Applications of Categories*, vol. 33, pp. 1158–1222, 2018.
- [17] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation* (Monographs in Theoretical Computer Science (EATCS Series)). Springer, 2006.

Revision History

Version	Date	Changes
1.0.0	March 2026	Initial release